REPUBLIC OF TUNISIA
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology

Graduation Project

To obtain the

National Engineering Diploma

In Applied Sciences and Technology

Specialty : Software Engineering

Design, Implementation, and Pipeline Integration of a Smoke Test Suite as a CI/CD Quality Gate for a Multi-Universe SaaS Platform

---

Presented by :

Ahmed SILINI

Hosted By

LittleBig Connection

Defended on the 9th of June 2026 in front of a jury composed of :

Mr. Emir DAMERJI	President of the jury
Mrs. Lilia SFAXI	Reviewer
Mrs. Hajer AKAICHI	Supervisor at the Company
Mrs. Sana HAMDI	Supervisor at INSAT

University year : 2025 / 2026

REPUBLIC OF TUNISIA
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology

Graduation Project

To obtain the

National Engineering Diploma

In Applied Sciences and Technology

Specialty : Software Engineering

Design, Implementation, and Pipeline Integration of a Smoke Test Suite as a CI/CD Quality Gate for a Multi-Universe SaaS Platform

---

Presented by

Ahmed SILINI

Hosted By :

LittleBig Connection

Supervisor at the Company Mrs. Hajer AKAICHI	Supervisor at INSAT Mrs. Sana HAMDI
Date :	Date :

University year : 2025 / 2026

Dedication

To my parents, Mohamed and Hanen,

for the years of patience, the quiet sacrifices, and the trust that made this path possible.

To my sisters, Amal and Ahlem,

for being the steady company through every long night and every small victory.

To my friends and to those who, from near or far, kept asking how the work was going,

your encouragement carried more weight than you know.

To everyone who taught me, formally or in passing,

this work bears traces of all of you.

Ahmed Silini

Acknowledgements

This project would not have taken its current form without the support and guidance of the people around me, whom I wish to thank here.

I am deeply grateful to my company supervisor at LittleBig Connection, **Hajer Akaichi**, QA Manager, for the trust placed in me from the first week, for the weekly reviews that kept every deliverable on track, and for fully integrating me into the QA team rather than treating the internship as a parallel track. The feedback on each iteration pushed the work to a higher standard than the one I would have settled for.

I also owe a thank-you to my company mentor, **Zied Klouz**, Senior QA Engineer, for the availability and technical guidance that helped clear the obstacles that arose during implementation.

I am particularly grateful to my academic supervisor at the National Institute of Applied Sciences and Technology (INSAT), **Dr. Sana Hamdi**, for the sustained attention she gave to this project throughout the internship. Her feedback on each submitted deliverable was precise and consistently pushed me toward clearer thinking and stronger writing. The structure and rigour this report has achieved would not have been possible without her guidance.

My thanks extend to the entire Quality Assurance team at LittleBig Connection, whose patience with my questions, willingness to share internal context, and reviews of early iterations shaped both the project and my understanding of how a production QA practice operates day to day.

I would like to thank the members of the jury, **Dr. Emir Damerji** and **Dr. Lilia Sfaxi**, for the time invested in reading and evaluating this report and for the opportunity to defend it.

Finally, to the faculty and staff of INSAT: thank you for the body of knowledge accumulated over five years of engineering studies that underpins every page of this work.

Ahmed Silini

Abstract

LittleBig Connection runs a workforce management platform used by enterprises, suppliers, and freelancers across four distinct application universes. The team's automated regression suite (976 test cases, Robot Framework, SeleniumLibrary) had grown into a liability: over eight hours to run, an unmeasured flakiness rate, and tests that silently depended on each other's data. A green pipeline was no longer trustworthy.

This report describes a smoke test suite built to address that problem. Running as the first stage of the GitLab CI/CD pipeline, it blocks the regression suite the moment a critical path breaks, instead of letting a broken build propagate into an eight-hour run. The suite uses Robot Framework with Browser Library, which uses Playwright as its browser engine, so the team could keep writing in keyword syntax without splitting the codebase into two languages.

Eighteen tests cover the twenty priority-one user journeys across Auth, Admin, Client, and Supplier. The Auth end-to-end test bundles admin login, user login, and log-out into one execution, while every other domain maps one test to one journey. Each test owns its data, cleans up after itself, and can run in any order. On pre-production, all eighteen finish in about two minutes in parallel mode, well inside the ten-minute target, and about 2.9× faster than the matched Selenium paths. Failure evidence is immediate: video, HAR network traces, and screenshots land in the artefact store without any extra configuration. A rerun budget absorbs the transient noise that was previously mis-read as product defects. After each run, an Adaptive Card reaches the QA team's Microsoft Teams channel; a GitLab Pages dashboard tracks execution time, flakiness, coverage, and test independence over time.

Keywords: Robot Framework, Browser Library, Playwright, smoke testing, end-to-end testing, CI/CD, GitLab, quality gate, test flakiness, test independence, page object model, DevOps.

Résumé

LittleBig Connection opère une plateforme de gestion de la main-d'œuvre étendue qui réunit entreprises clientes, fournisseurs de services et consultants indépendants au sein de quatre univers applicatifs. La suite de régression de l'équipe QA (976 cas, Robot Framework et SeleniumLibrary) était devenue un frein : plus de huit heures d'exécution, un taux de flakiness non mesuré, et des tests implicitement couplés aux données laissées par leurs prédécesseurs. Un pipeline vert ne garantissait plus la fiabilité du produit.

Ce rapport décrit une suite de tests *smoke* conçue pour résoudre ce problème. Positionnée en première étape du pipeline GitLab CI/CD, elle bloque la suite de régression dès qu'un parcours critique échoue, au lieu de laisser un build défaillant s'engager dans huit heures d'exécution inutile. La suite repose sur Robot Framework avec la *Browser Library*, qui exploite Playwright comme moteur navigateur : l'équipe conserve ainsi la syntaxe clé qu'elle maîtrise, sans fragmenter la base de code en deux langages.

Dix-huit tests couvrent les vingt parcours utilisateurs prioritaires des univers Auth, Admin, Client et Fournisseur. Le test *end-to-end* Auth regroupe en une seule exécution la connexion administrateur, la connexion utilisateur et la déconnexion, tandis que chaque autre domaine associe un test à un parcours. Chaque test gère ses propres données, fait son nettoyage, et peut s'exécuter dans n'importe quel ordre. Sur la pré-production, les dix-huit tests terminent en environ deux minutes en mode parallèle, bien en deçà de la cible de dix minutes. Le gain est d'environ 2.9× par rapport aux parcours Selenium équivalents. Les preuves d'échec sont immédiates : vidéo, traces réseau HAR et captures d'écran sont déposées dans les artefacts sans configuration supplémentaire. Un budget de relance absorbe le bruit transitoire qui était auparavant interprété à tort comme des défauts produit. Après chaque exécution, une carte adaptative est envoyée sur le canal Microsoft Teams de l'équipe QA ; un tableau de bord GitLab Pages suit le temps d'exécution, la flakiness, la couverture et l'indépendance des tests au fil du temps.

Mots-clés : Robot Framework, Browser Library, Playwright, tests *smoke*, tests end-to-end, CI/CD, GitLab, barrière qualité, flakiness, indépendance des tests, page object model, DevOps.

Contents

General Introduction **1**

1 General Context **3**

1.1 Host Organization: LittleBig Connection 4

1.1.1 Company Overview and Business Sector 4

1.1.2 The Four User Universes 4

1.1.3 IT Organization and Release Management Process 5

1.2 Project Presentation 6

1.2.1 Project Context 6

1.2.2 Existing State 7

1.2.3 Problem Statement 7

1.3 Proposed Solution 8

1.4 Work Methodology 9

1.4.1 Waterfall 9

1.4.2 Agile and Scrum 9

1.4.3 Iterative and Incremental Development 9

1.4.4 Methodology Comparison and Adopted Choice 10

1.5 Project Phases and Timeline 12

2 State of the Art: Test Automation in Modern CI/CD Pipelines **15**

2.1 Software Testing Fundamentals 16

2.1.1 Test Levels 16

2.1.2 Test Types 16

2.1.3 Smoke Testing 17

2.2 Test Automation Concepts 17

2.2.1 The Test Automation Pyramid 17

2.2.2 Test Flakiness: Definition, Causes, and Measurement 18

2.2.3 Test Independence and the Coupling Anti-Pattern 19

2.2.4 Test Data Management Strategies 19

2.3 Modern DevOps Practices and Test Automation 20

2.3.1 Shift-Left Testing and the Fail-Fast Principle 20

2.3.2 Automated Quality Gates in CI/CD Pipelines 21

2.4 Browser Automation Frameworks 21

2.4.1 Selenium / WebDriver . . . 21
2.4.2 Cypress . . . 22
2.4.3 Robot Framework . . . 23
2.4.4 Playwright . . . 24
2.4.5 Robot Framework with Playwright . . . 25
2.4.6 Comparative Analysis and Technology Choice . . . 26

3 Requirements Analysis and System Design 28
3.1 System Requirements . . . 29
3.1.1 Actors Identification . . . 29
3.1.2 Functional Requirements . . . 29
3.1.3 Non-Functional Requirements . . . 30
3.1.4 Risk Register . . . 31
3.2 Critical Path Analysis . . . 31
3.2.1 Selection Criteria for the Smoke Suite . . . 32
3.2.2 Critical User Journeys per Domain . . . 32
3.3 System Modeling and Design . . . 33
3.3.1 Use Case Diagram . . . 33
3.3.2 Sequence Diagram: Running the Smoke Suite . . . 34
3.3.3 Layered Architecture . . . 35
3.3.4 Package Dependency Diagram . . . 37
3.3.5 Pipeline Design . . . 39

4 Implementation of the Smoke Test Framework 41
4.1 Development Environment and Tooling . . . 42
4.2 Framework Skeleton . . . 44
4.2.1 Project Structure . . . 44
4.2.2 Shared Resources and Utilities . . . 44
4.2.3 Page Object Resources and Universe Aggregators . . . 45
4.3 Authentication and Session Management . . . 45
4.4 Test Data Strategy . . . 46
4.4.1 Centralised Fixtures Module . . . 46
4.4.2 Read-Only Tests . . . 46
4.4.3 Write Tests . . . 47
4.5 Smoke Test Implementation per Domain . . . 47
4.5.1 Auth Domain . . . 48
4.5.2 Client Domain . . . 48
4.5.3 Supplier Domain . . . 49
4.5.4 Admin Domain . . . 49
4.6 Code Quality and Linting . . . 50

4.7 Metrics Dashboard . . . 51
4.7.1 Stack and Data Contract . . . 51
4.7.2 Visual Identity and Theming . . . 51

5 Deployment and CI/CD Integration 53
5.1 Containerisation and Test Image . . . 54
5.2 Pipeline Implementation . . . 54
5.2.1 Merge-Request Pipeline . . . 55
5.2.2 Web and Scheduled Pipeline . . . 55
5.3 Quality Gate Configuration . . . 56
5.3.1 Fire-and-Forget Dispatch . . . 58
5.3.2 Containment of False Positives . . . 58
5.4 Trigger Strategy and Execution Modes . . . 58
5.4.1 Parallel and Sequential Execution Modes . . . 59
5.4.2 Flakiness Mitigation through Reruns . . . 60
5.4.3 Environment Routing . . . 60
5.5 Reporting Artefacts . . . 61
5.6 Traceability and Notification Layer . . . 62
5.6.1 Notification Stage . . . 62
5.6.2 Adaptive Card Layout . . . 62
5.6.3 Data Sources and Triggers . . . 63
5.6.4 Downstream Hand-off . . . 63
5.7 Metrics Dashboard Deployment . . . 63
5.7.1 Job Steps . . . 63
5.7.2 Stability Probe Pipeline . . . 64

6 Results and Evaluation 66
6.1 Evaluation Methodology . . . 67
6.2 Execution Time Benchmark . . . 67
6.2.1 Metric Definition . . . 68
6.2.2 Per-Domain Comparison . . . 68
6.2.3 Parallel versus Sequential Mode . . . 70
6.2.4 Evolution Across Pipeline Runs . . . 71
6.3 Flakiness and Test Independence . . . 71
6.3.1 Metric Definitions . . . 71
6.3.2 Probe Design . . . 72
6.3.3 Flakiness and Independence Results . . . 72
6.4 Gate Effectiveness and Coverage . . . 73
6.4.1 Critical Path Coverage . . . 73
6.4.2 Gate Effectiveness . . . 74

6.4.3 Remaining Requirements Verification 77
6.5 Limitations and Threats to Validity 77

General Conclusion and Perspectives **79**

Bibliography **81**

Appendices **83**
Appendix A: Critical User Journey Catalogue 83
Appendix B: Per-Domain Design Class Diagrams 86
Appendix C: Repository Layout 89

List of Figures

1.1 LittleBig Connection and Mantu Group 4
1.2 LB Tech department organisation 5
1.3 Smoke gate ahead of the regression run 9
1.4 The Scrum framework 9
1.5 The iterative and incremental model 10
1.6 Project Jira Kanban board (screenshot) 11
1.7 Cumulative flow diagram from Jira Reports (screenshot) 12
1.8 Project Gantt chart 13

2.1 The test automation pyramid [3] 18
2.2 Selenium / WebDriver architecture [9] 22
2.3 Cypress architecture [10] 23
2.4 Robot Framework architecture [11] 24
2.5 Playwright architecture [12] 25

3.1 Use case diagram of the framework 34
3.2 Sequence diagram: running the smoke suite 35
3.3 Layered architecture of the framework 36
3.4 Package Dependency diagram of the framework 38
3.5 Auxiliary pipeline activity diagrams: merge-request gate and stability probe 39
3.6 Web and scheduled pipeline activity diagram 40

4.1 LittleBig Connection dashboard palette 52

5.1 Pipeline launch form on GitLab 58
5.2 Pipeline graphs in parallel and sequential modes 60
5.3 Stability-probe pipeline on GitLab 64
5.4 Weekend probe schedules on GitLab 65

6.1 Metrics dashboard in web view 67
6.2 Metrics dashboard in mobile view 67
6.3 Client domain runtime, Selenium against the smoke suite 69
6.4 Supplier domain runtime, Selenium against the smoke suite 69
6.5 Admin domain runtime, Selenium against the smoke suite 69
6.6 Playwright Inspector during local debugging 71

6.7 Execution-time card . . . 71
6.8 Flakiness card . . . 73
6.9 Test-independence card . . . 73
6.10 Critical path coverage card . . . 74
6.11 Failed smoke run blocking the regression trigger . . . 74
6.12 Rerun mechanism absorbing a transient failure . . . 75
6.13 QA team Microsoft Teams channel . . . 75
6.14 Adaptive Card in three states . . . 76
6.15 Downstream regression pipeline card in its own Teams channel . . . 76

A.1 Title page of D1. . . . 83
B.1 Design class diagram: Auth domain (PoC) . . . 86
B.2 Design class diagram: Admin domain . . . 86
B.3 Design class diagram: Client domain . . . 87
B.4 Design class diagram: Supplier domain . . . 88

List of Tables

1.1 The four user universes of the LBC platform 5
1.2 Existing Robot Framework suite scale 7
1.3 Smoke suite targets against the existing suite 8
1.4 Comparison of the three methodologies considered 10

2.1 The four test levels 16
2.2 The four test types 17
2.3 Test data management strategies 20
2.4 Browser automation framework comparison 26

3.1 Functional requirements 30
3.2 Non-functional requirements 30
3.3 Project risk register 31
3.4 Smoke suite test count and CUJ coverage per domain 33

4.1 Development tooling and the role of each component 43

5.1 Pipeline stages, jobs, and activation rules 55
5.2 Pipeline inputs exposed on the trigger form 59
5.3 Run artefacts and their consumers 61
5.4 Adaptive Card sections in display order 62

6.1 Per-domain runtime, Selenium against RF with Browser Library. Auth, the suite's one login test, has no scope-matched Selenium counterpart, so it is shown for reference and excluded; the matched total covers the other 17 of the 18 tests. 68
6.2 Smoke suite total runtime by execution mode 70
6.3 Flakiness and independence at three reference probes 72
6.4 Critical path coverage per domain and across priority tiers 74
6.5 Verification evidence for the remaining requirements 77

A.1 Auth domain critical user journeys, extract from D1 v1.1 84
A.2 Client universe critical user journeys, extract from D1 v1.1 84
A.3 Supplier universe critical user journeys (including Freelancer flows), extract from D1 v1.1 85
A.4 Admin universe critical user journeys, extract from D1 v1.1 85

List of Acronyms

API	Application Programming Interface
BDC	Bon de Commande (purchase order)
CDP	Chrome DevTools Protocol
CI/CD	Continuous Integration / Continuous Delivery
CRA	Compte Rendu d'Activité (activity report)
CSS	Cascading Style Sheets
CUJ	Critical User Journey
DOM	Document Object Model
DSL	Domain-Specific Language
E2E	End-to-End
FR	Functional Requirement
gRPC	google Remote Procedure Call
HAR	HTTP Archive
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
INSAT	Institut National des Sciences Appliquées et de Technologie
JS	JavaScript
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LBC	LittleBig Connection
MR	Merge Request
NFR	Non-Functional Requirement
NRT	Non-Regression Tests
POM	Page Object Model
QA	Quality Assurance
RF	Robot Framework
RFP	Request for Proposal
SDK	Software Development Kit
SSOT	Single Source of Truth
TS	TypeScript
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
XML	eXtensible Markup Language

General Introduction

Modern web platforms rely on Continuous Integration and Continuous Delivery (CI/CD) pipelines to release changes several times a day. At that cadence, automated end-to-end testing is what stands between a developer's commit and the customer. Each pipeline runs the suite, the suite passes or stops the deployment, and the team trusts that verdict to gate the next step. The reliability of that gate decides whether continuous delivery is safe or merely fast. As suites grow, runtime, flakiness, and hidden coupling between tests slowly erode the trust placed in them. A multi-hour pipeline whose red status no one is sure how to interpret has stopped acting as a gate, even when most of its tests still pass.

LittleBig Connection (LBC) is in exactly that position. Its platform serves enterprises, suppliers, and freelancers across four distinct application universes, with any one of them able to break independently. The Quality Assurance (QA) team owns the regression suite that protects all four. It contains 976 test cases built on Robot Framework with SeleniumLibrary, runs through the GitLab pipeline, and reports back to Jira via Xray. The suite has grown into a liability. A full run takes over eight hours, the flakiness rate has never been measured, and several tests rely on data created by earlier ones. A single failure early in the run cascades through unrelated tests, and the team has learned to treat red builds as noise rather than as signals. The pipeline still runs, but the team no longer relies on its verdict to decide whether a release is safe.

The contribution of this work is a smoke test suite that runs in front of the existing regression suite, not in place of it. The suite covers the priority-one critical user journeys across the four universes, completes in under ten minutes, and blocks the regression run when a critical path fails. To keep the QA codebase under a single authoring convention, we kept Robot Framework as the keyword layer and replaced the underlying engine with Playwright through its Browser Library binding. The two suites therefore share one language, while the smoke layer benefits from a faster, event-driven browser protocol and from native video and HTTP Archive (HAR) network trace capture. Each run posts a Microsoft Teams Adaptive Card to the QA channel. A GitLab Pages dashboard records the four evaluation metrics (execution time, flakiness, critical path coverage, and test independence) across consecutive pipelines, so drift becomes visible over time.

The report is organised into six chapters. Chapter 1 introduces LBC and its QA team, formalises the limitations of the existing suite, and describes the iterative and incremental

methodology adopted across the five-month internship. Chapter 2 grounds each design decision in the state of the art. It covers testing levels, the automation pyramid, and the structural causes of flakiness. It then compares five candidate browser-automation stacks against the project's hard constraints, and that comparison leads to the technology choice. Chapter 3 translates that comparison into a specification, with functional and non-functional requirements, the catalogue of priority-one critical user journeys, and the framework's layered architecture. Chapter 4 then covers the implementation, from the project skeleton and the authentication fast path through the per-domain test packs and the metrics dashboard. Chapter 5 documents the deployment side. It covers the Docker test image, the GitLab pipeline shapes, the gating logic between the smoke run and the downstream regression trigger, and the notification layer. Chapter 6 measures the delivered suite against the targets fixed in Chapter 1 and discusses the threats to validity that frame those results. The General Conclusion then revisits the contribution and outlines the next steps for the team.

Chapter 1

General Context

Summary

1.1 Host Organization: LittleBig Connection
1.2 Project Presentation
1.3 Proposed Solution
1.4 Work Methodology
1.5 Project Phases and Timeline

Introduction

This chapter locates the project in its organisational and technical context. The first sections cover LBC's business model and the structure of the IT department that hosts the QA team. The next section documents the existing Robot Framework suite and the failure modes that motivated this work. The final sections introduce the proposed solution, the iterative methodology, and the five-month project timeline.

1.1 Host Organization: LittleBig Connection

1.1.1 Company Overview and Business Sector

Founded in 2013, LittleBig Connection (LBC) operates in the **Extended Workforce Management** sector. Enterprises use its platform to find, contract, and manage external service providers: consulting firms, staffing agencies, and independent professionals, all through a single interface instead of separate procurement tools. LBC has around **400 employees**, operates in over **50 countries**, and is approaching half a billion euros in annual revenue.

The client base is broad: banks and insurers, telecoms operators, energy groups, and industrial conglomerates all use the platform. Each sector brings its own compliance constraints and procurement workflows, which the platform must support without fragmenting the user experience.

LBC belongs to the **Mantu Group**, a professional services and technology group, as shown in Figure 1.1. Within Mantu, LBC supplies the platform that other group entities rely on when they need to place or manage external talent.



Figure 1.1: LittleBig Connection and Mantu Group

The platform sits at the intersection of HR management and procurement. Client organisations consolidate workforce sourcing and contract management through a single interface. This has a direct consequence for quality assurance: the platform must be tested not as a single application, but as a set of interconnected interfaces serving actors with fundamentally different roles and permissions.

1.1.2 The Four User Universes

LBC refers to its four user-facing environments as *universes*. Each maps to a distinct category of actors with a dedicated feature set. Table 1.1 summarises each universe and its role within the platform.

Universe	Description
Admin	Used by internal LBC team members. Exposes back-office tools for user management, platform configuration, and operational oversight across all other universes.
Client	Used by enterprise representatives. They publish service requests, evaluate and select providers, manage contracts, and approve invoices. It is the entry point for the core business flow.
Supplier	Used by companies (staffing agencies and consulting firms) that receive service requests, submit proposals, manage their consultants, and handle invoicing on their side of the contract.
Freelancer	Used by independent consultants managing their profile, missions, and deliverables. Lighter than the Client and Supplier universes but integral to the end-to-end delivery chain.

Table 1.1: The four user universes of the LBC platform

These universes are **not independent**. A service request created by a Client must be visible and actionable by a Supplier. A signed contract must be reflected on both sides. Many business flows therefore cross universe boundaries and can only be validated by coordinating actions across multiple user types.

1.1.3 IT Organization and Release Management Process

The technical department at LBC operates under the **LB Tech** umbrella. Figure 1.2 shows how it is organised.

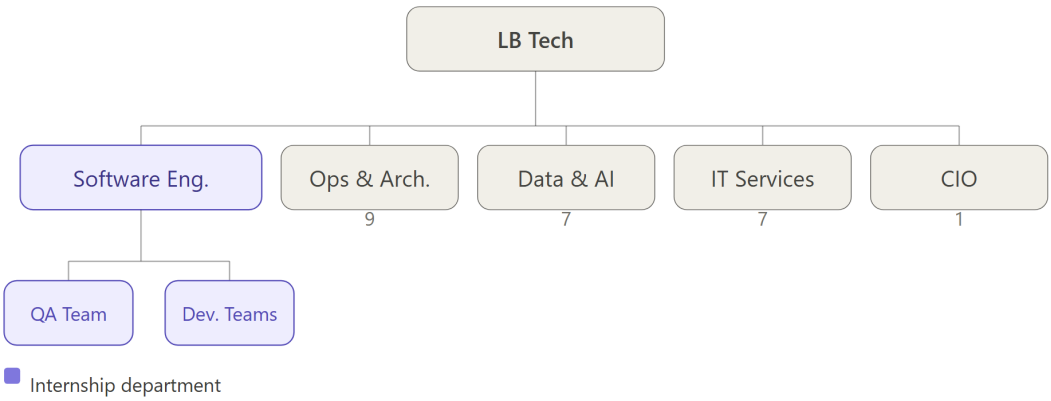


Figure 1.2: LB Tech department organisation

LB Tech is divided into five groups: Software Engineering (Eng.), Operations and Architecture (Ops & Arch.), Data and Artificial Intelligence (Data & AI), IT Services, and the Chief Information Officer (CIO) office. The numbers shown in the chart indicate the headcount per group (9 for Ops & Arch., 7 each for Data & AI and IT Services). Software Engineering is further divided into two sub-teams: the Quality Assurance (QA) Team and the Development (Dev.) Teams. These two sub-teams constitute the internship host department, shown in blue in the diagram.

The QA team is embedded across business squads organised by product domain. Its engineers work alongside developers and product managers throughout the delivery cycle. They are not split into manual and automation specialists. Within a squad, the same engineers run acceptance flows on new features, maintain the automated regression suite, and triage daily pipeline failures. Test case documentation and execution results are tracked in Jira and Xray, which link each test result to its source requirement and maintain full traceability from specification to execution.

Releases follow a structured workflow. When development tasks are complete, the QA team runs functional validation. Automated suites then execute within the GitLab **Continuous Integration and Continuous Delivery (CI/CD)** pipeline to detect regressions before the build is promoted. Any failure at this stage is resolved and revalidated before the release proceeds. The Ops & Arch. team's DevOps engineers then handle deployment across environments and run additional checks to confirm stability before the new version reaches production.

This process makes every pipeline failure a direct blocker. The team that wrote the tests is the same team that must fix them before delivery can continue, so the speed and reliability of the automated gate matter as much as its coverage.

1.2 Project Presentation

1.2.1 Project Context

The internship took place within the Software Engineering department at LBC, hosted by the QA team, from February to June 2025. As described in Section 1.1, the QA team covers both manual validation and automated regression work. The internship focused specifically on the automation side, with the mission of designing and implementing a smoke test suite for the platform.

A smoke suite is a fast automated health check: it runs before the full regression cycle and stops it if a critical path is broken. The scope covered the full automation lifecycle, from requirement analysis and framework architecture design through test implementation,

CI/CD integration, and a metrics dashboard tracking the four evaluation criteria defined at the project's outset.

1.2.2 Existing State

The existing automated test suite covers all four universes with a substantial volume of test cases. Table 1.2 reports its scale.

Scope	Test Cases	Test Files
All four universes (Admin, Client, Supplier, Freelancer)	**976**	**196**

Table 1.2: Existing Robot Framework suite scale

The suite runs within the GitLab CI/CD pipeline, either on demand or on a nightly schedule as the team's **Non-Regression Tests (NRT)** job. It is not triggered on every push or merge request: an eight-hour execution cycle would dominate the developer feedback loop if it were. Results feed back into Jira through Xray.

1.2.3 Problem Statement

The platform's cross-universe interdependency is itself a challenge for test automation. It inflates both the surface each test exercises and the failure modes that surface during a run. The full suite takes **over eight hours** to complete. There is no lightweight validation step that can quickly confirm whether the platform is operational after a code change, so the team must either wait for the next nightly run or trigger the full regression pipeline manually.

A portion of the tests fail intermittently for reasons unrelated to actual defects. Timing issues, implicit waits, and transient network conditions cause tests to fail and then pass again with no code change. The flakiness rate is not formally measured, which makes it impossible to distinguish real failures from noise in pipeline results.

Some tests also depend on data created by earlier tests or assume the presence of pre-existing platform state. This coupling prevents parallel execution, makes it difficult to investigate a single failing test in isolation, and causes failure cascades where one broken test brings down unrelated ones downstream.

When a test fails, the diagnostic information available is limited. SeleniumLibrary does not natively capture video or network traces. Screenshots exist but are collected incon-

sistently. Reproducing a failure typically requires re-running the test locally, which adds time to an already slow feedback loop.

These limitations compound each other. A multi-hour pipeline with an unmeasured flakiness rate trains the team to distrust automated results, so a real regression can hide among failures that everyone assumes are flaky. Without a fast validation gate, a critical platform breakage may not surface until hours after it is introduced. The outcome is a slower development cycle, a heavier triage burden, and elevated release risk.

1.3 Proposed Solution

We propose a smoke test suite that runs alongside the existing Robot Framework regression suite rather than replacing it. It adds a **fast validation layer** that runs first in the pipeline and **blocks the regression suite** if a critical path fails.

A smoke suite validates only the **most critical user journeys** across the platform. Its role is a health check: confirm the platform is operational before committing to an eight-hour regression cycle. Table 1.3 compares the current suite with the targets we set for the new one.

Metric	RF Suite (current)	Smoke Suite (target)
Execution time	$>$ 8 hours	$<$ 10 minutes
Flakiness rate	Not tracked	$<$ 2%
Test independence	Partial	100%
CI gate	On-demand / nightly only	Smoke-first on web/schedule

Table 1.3: Smoke suite targets against the existing suite

The new suite is built on a modern browser-automation engine while remaining authored in the same keyword syntax the team already uses, so the QA codebase does not split across two languages or conventions. The full framework comparison that led to this choice is presented in Chapter 2.

The resulting pipeline places the smoke suite as the **first stage** on triggered runs. The full Robot Framework regression suite only runs if the smoke suite passes; Figure 1.3 shows the gate.

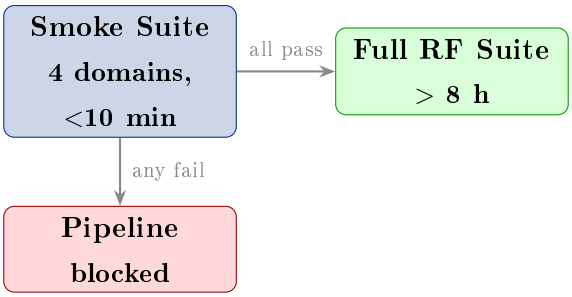


Figure 1.3: Smoke gate ahead of the regression run

1.4 Work Methodology

1.4.1 Waterfall

The Waterfall model executes a project as a strict sequence of phases (requirements, design, implementation, testing, and delivery), each fully completed before the next begins. It cannot absorb mid-project discoveries: a technical finding in a later phase has no clean resolution path without disrupting the entire schedule.

1.4.2 Agile and Scrum

Scrum organises work into fixed-length sprints built around team ceremonies: sprint planning, daily standups, reviews, and retrospectives. Applied to a solo internship, this overhead brings little return, since most of its value comes from team coordination rather than individual task management. Figure 1.4 depicts the Scrum sprint cycle.

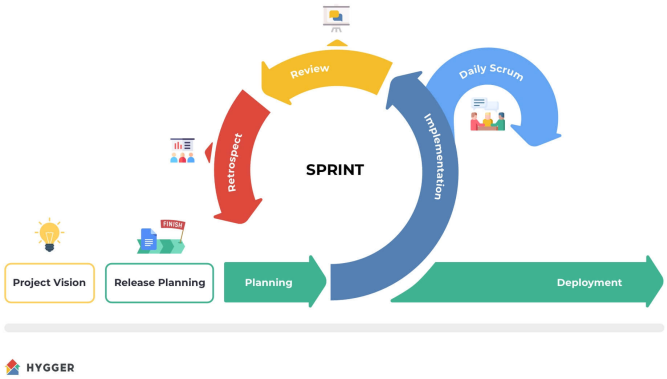


Figure 1.4: The Scrum framework

1.4.3 Iterative and Incremental Development

Iterative and incremental development divides a project into phases, each producing a validated deliverable that the next builds upon. Unlike Waterfall, it absorbs mid-project

feedback without abandoning the overall plan; unlike Scrum, it imposes no ceremony overhead. Figure 1.5 illustrates this cyclical structure.

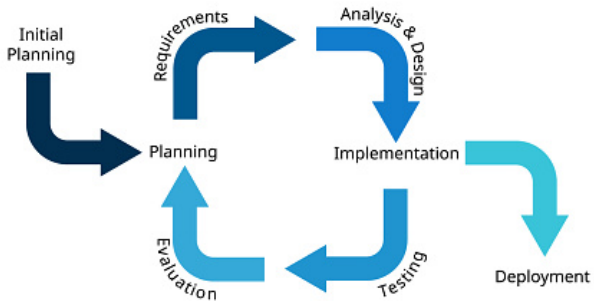


Figure 1.5: The iterative and incremental model

1.4.4 Methodology Comparison and Adopted Choice

The three approaches differ on adaptability, ceremony cost, intermediate deliverable cadence, and fitness for a solo internship. Table 1.4 compares them across the criteria most relevant to this project.

Criterion	Waterfall	Scrum	Iterative & Incremental
Adaptability to change	Low	High	High
Ceremony overhead	Low	High	Low
Intermediate deliverables	No	Every sprint	Every phase
Suitable for solo internship	Partially	No	**Yes**
Traceability	High	Medium	High

Table 1.4: Comparison of the three methodologies considered

The iterative and incremental approach fits this project directly. In practice it is an agile method stripped of the ceremonies that bring little return on a solo internship. We run it as a **Kanban workflow on Jira**. The work splits into six phases, and the same board records both the phase deliverables and the day-to-day task flow.

A separate **Jira backlog** sits alongside the active board and documents each iteration's scope and target deadline. New work is groomed there before it is pulled onto the active board, which keeps the deadline trail visible without crowding the in-progress view. We track progress on two cadences. A weekly 30-minute sync with the team lead surfaces

blockers on the company side. A **monthly progress report** to the academic supervisor records the current phase, tasks completed that month, technical learnings, upcoming work, and overall status. All changes are versioned in GitLab with descriptive commit messages.

The active Jira board is reproduced as a screenshot in Figure 1.6, with columns for *To Do*, *In Progress*, *In Review*, and *Done*, and tasks tagged by universe and category.

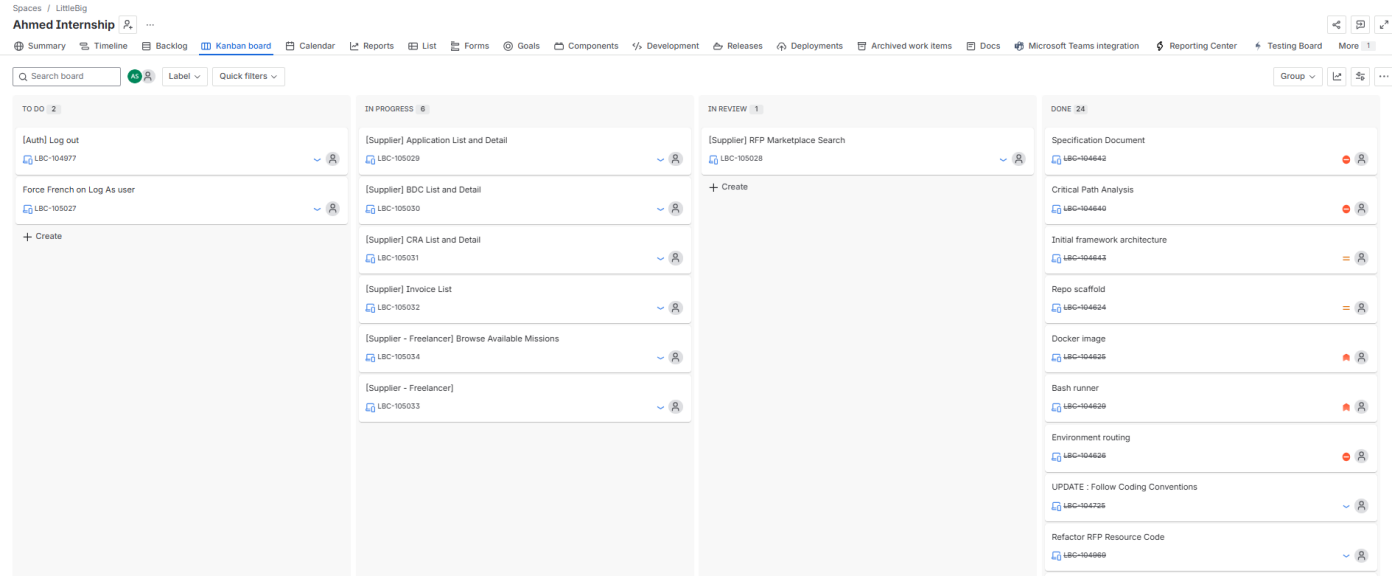


Figure 1.6: Project Jira Kanban board (screenshot)

Aggregated over the full internship, the same task history yields the cumulative flow diagram in Figure 1.7, taken from the **Reports** section of the same Jira project. It shows how work moved through the workflow states over time. Spikes at the top of the orange *Backlog* band mark the start of successive iterations: each new phase opens with a fresh batch of scoped tasks, which the team then pulls into progress, review, and done.

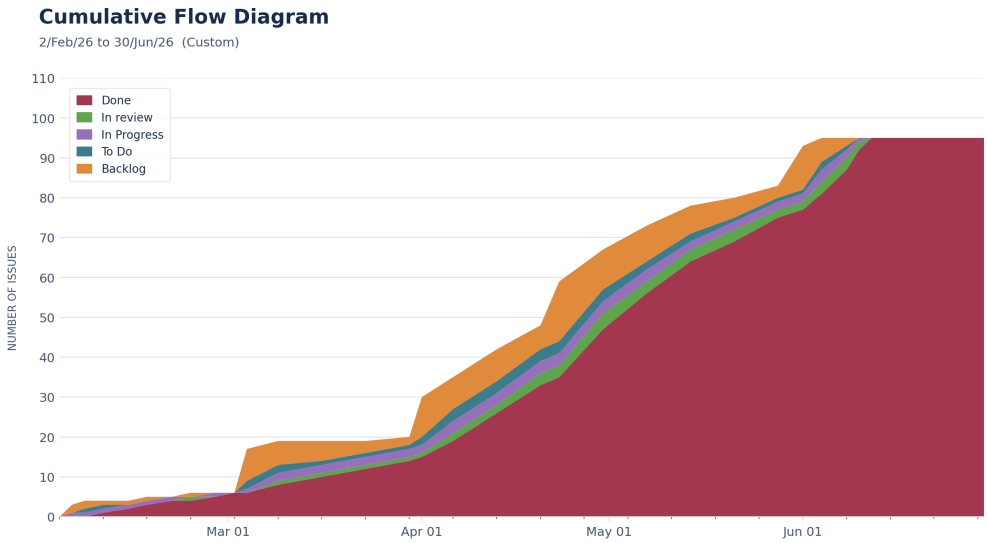


Figure 1.7: Cumulative flow diagram from Jira Reports (screenshot)

1.5 Project Phases and Timeline

The project is structured into six phases, each producing one or more deliverables and bounded by a review rather than a fixed calendar date. The phases distribute across the five-month internship from February to June, their duration set by scope and feedback. The internship report (**D11**) runs alongside every phase: we draft it continuously throughout the project and consolidate it in the final phase. It therefore spans every bar in the Gantt rather than belonging to any single one. Figure 1.8 presents the resulting schedule.

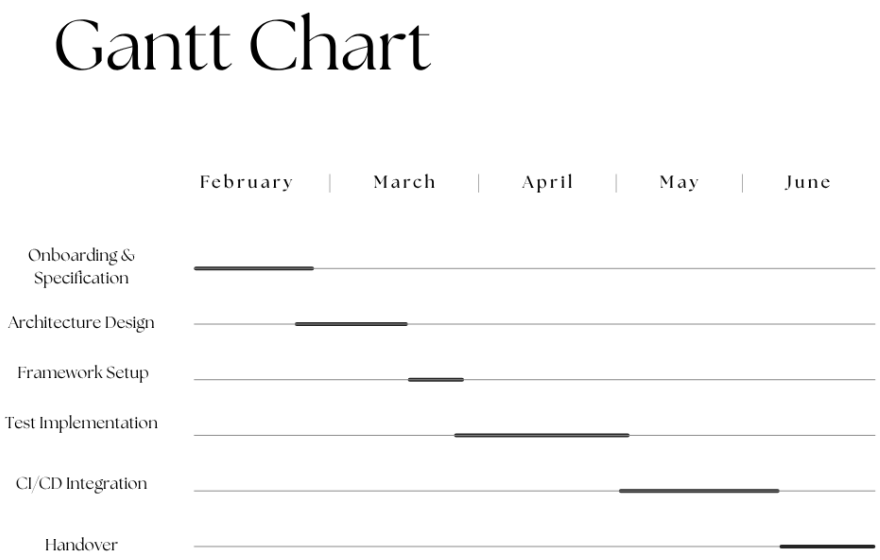


Figure 1.8: Project Gantt chart

At each review, we present the phase deliverables to the team lead. The exercise yields two outcomes: confirmation that the phase output is fit for purpose, and early sight of any unexpected findings (technical constraints, scope adjustments, environmental issues) before they have time to compound.

Increment 1: Onboarding & Specification (D0, D1, February). The project opens with team onboarding and a specification document (**D0**) that frames the business problem, documents the current state of the Robot Framework suite, and articulates the smoke suite objectives in measurable terms. This document is the written contract between the team lead and the intern about what constitutes success. A critical path analysis (**D1**) then identifies which user journeys across all four universes are genuinely critical: those that, if broken, would prevent the platform from functioning. The catalogue is consulted throughout implementation, so every test case covers one of these critical paths and scope creep into peripheral tests is prevented from the start.

Increment 2: Architecture Design (D2, late February). A framework architecture design (**D2**) translates the requirements into a layered architecture. It specifies how page resources, utility modules, and test files will be organised, how authentication will be cached to avoid redundant logins, and how the framework will support both local and CI execution. Producing this design before any code is written settles these concerns up front, so implementation starts from a known structure.

Increment 3: Framework Setup (D3, mid March). The framework skeleton (**D3**)

realises the architecture from D2 in code. It establishes the project structure, the configuration strategy, and the reusable authentication layer that subsequent tests will build upon. Completing the skeleton before any business test keeps the codebase style consistent from the start and avoids retrofitting the framework around code that already exists.

Increment 4: Test Implementation (D4, D5, D6, late March to mid April). Smoke test implementation proceeds one universe at a time. The Client pack (**D4**) comes first: it is the highest-profile user group and the entry point to the critical contract flow. The Supplier pack (**D5**) follows, covering the service provider side of the core workflows along with the two Freelancer journeys. The Admin pack (**D6**) closes the increment with the back-office journeys. Sequencing the universes rather than tackling them in parallel surfaces timing and authentication issues one domain at a time, before they multiply across the codebase.

Increment 5: CI/CD Integration & Traceability (D7, D8, D9, mid April to late May). The CI/CD gate (**D7**) integrates the smoke suite into the GitLab pipeline, configures the smoke-gates-regression pattern, and handles parallel execution, reruns, and artifact handling. The traceability layer (**D8**) adds Microsoft Teams notifications, Playwright trace collection, and the metrics dashboard. The benchmark deliverable (**D9**) then quantifies the suite against the four evaluation metrics and establishes the baseline the team will track going forward.

Increment 6: Handover (D10, D11 final consolidation, June). The migration roadmap (**D10**) outlines the forward path: finishing the legacy structural refactor, migrating the regression suite from Selenium to Playwright squad by squad, and optionally consolidating the smoke suite into the regression repository. The internship report (**D11**), drafted continuously throughout the project, is consolidated here. Together with the roadmap, it is the primary handover artefact for whoever maintains the suite next.

Conclusion

The limitations of the existing Robot Framework suite are not minor inconveniences. They directly affect the team's ability to release with confidence. Our smoke suite addresses these problems with a focused scope: validate the critical paths, run in under ten minutes, and block the pipeline when something is broken. The following chapters describe how we designed, built, and evaluated that solution.

Chapter 2

State of the Art: Test Automation in Modern CI/CD Pipelines

Summary

2.1 Software Testing Fundamentals
2.2 Test Automation Concepts
2.3 Modern DevOps Practices and Test Automation
2.4 Browser Automation Frameworks

Introduction

Before building the smoke suite, we need to ground each design decision in how modern CI/CD pipelines actually work. This chapter places smoke testing in the broader test hierarchy, examines the automation patterns that govern flakiness and test independence, surveys the DevOps practices that determine where automated tests sit in a pipeline, and compares the candidate browser-automation frameworks against the specific constraints of this project.

2.1 Software Testing Fundamentals

2.1.1 Test Levels

Software testing is organised into levels that differ by scope, target, and the stage of development at which they apply [1]. Standard classifications define four, summarised in Table 2.1.

Level	Focus
Component (unit)	Individual functions or classes verified in isolation, with no running infrastructure.
Integration	Whether components work correctly together: for instance, a service consuming an Application Programming Interface (API) or writing to a database as expected.
System	The complete, integrated application exercised against its specification. End-to-End (E2E) tests of full user flows, from the User Interface (UI) through the backend, sit here [2].
Acceptance	Fitness for release, usually judged from the user's point of view.

Table 2.1: The four test levels

The two higher levels work from specifications rather than source code, so they treat the software as a black box: they check external behaviour, not internal structure. This is the common case, not a strict rule: structural techniques can in principle apply at any level [1]. Component and integration testing, by contrast, typically draw on knowledge of the code.

Testing the LBC platform end to end requires the system level. Unit and integration tests cannot reach cross-universe behaviour. The existing Robot Framework (RF) regression suite operates at this level.

2.1.2 Test Types

Beyond levels, tests are classified by objective [1]. Table 2.2 sets out the four categories.

Type	What it checks
Functional	What the system does: whether a feature returns the expected output for a given input.
Non-functional	How well it behaves: qualities such as performance, security, and usability.
White-box	The internal structure rather than the external behaviour: it checks how much of the code a run actually exercises.
Change-related	Behaviour after a code change: confirmation testing re-checks a specific fix, while regression testing guards against breaking what already worked.

Table 2.2: The four test types

These objectives cut across the levels: any type can run at any level [1]. The smoke suite, for instance, is functional and change-related: it re-checks the platform after each deployment.

2.1.3 Smoke Testing

Smoke testing sits at the same system level as the regression suite but covers a radically reduced scope. Where the regression suite checks correctness across every path, the smoke suite checks only reachability and basic operability. It runs on the smallest set of critical paths, the ones whose failure would render the rest of the suite irrelevant. That scope reduction is what makes sub-ten-minute execution achievable. The two are not interchangeable: they answer different questions at different points in the pipeline.

2.2 Test Automation Concepts

2.2.1 The Test Automation Pyramid

The test automation pyramid, from Mike Cohn [3] and Martin Fowler [4], ranks automated tests by cost and speed rather than by scope. Its three layers, unit, service, and UI, map onto the lower test levels. The unit base is fast and cheap; the UI top is slow and costly to maintain, and it carries the system-level E2E tests defined above. The shape encodes one recommendation: invest heavily at the base and selectively at the top. Figure 2.1 shows the structure.

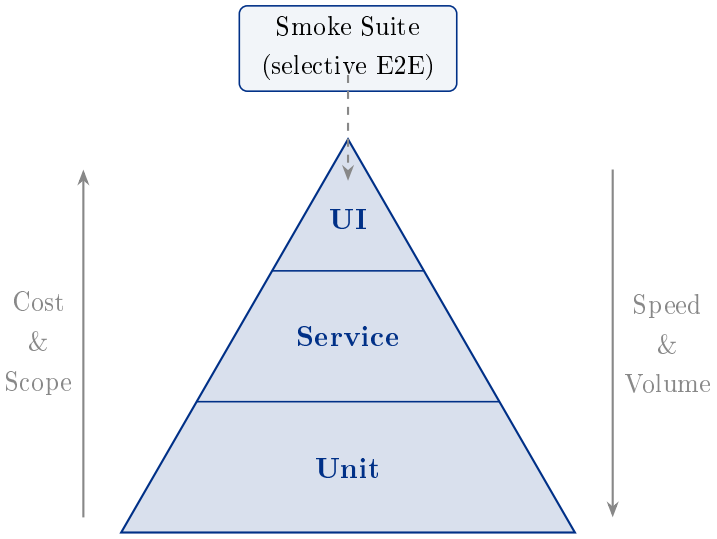


Figure 2.1: The test automation pyramid [3]

For this project, the lesson is that E2E tests must stay selective: they are expensive to write and run, so their count must stay low. A smoke suite is the extreme case, maximum coverage of critical paths with a minimum test count. The critical path analysis in Chapter 3 decides which user journeys earn a place in that set.

2.2.2 Test Flakiness: Definition, Causes, and Measurement

A flaky test passes in one run and fails in the next without any change to the system under test. The failure says nothing about the application; it is noise generated by the test itself or its execution environment. Flakiness is a corrosive problem, because it erodes the trust the suite depends on. Once a team learns to expect unreliable results, genuine failures get dismissed along with the noise, and the suite stops functioning as a quality gate.

Three categories of causes are relevant here:

- **Implicit timing assumptions** arise when a test triggers an action and immediately checks for a result, assuming the application responds within an unstated time window. In WebDriver-based tools this is structural: every browser command is a HyperText Transfer Protocol (HTTP) round-trip, and the browser state between commands is opaque to the test runner. Fixed waits are a common workaround, but they are either too short under load or wasteful under ideal conditions.

- **Shared mutable state** arises when tests rely on data produced by earlier tests, and one failure can cascade through dozens of unrelated results.

- **Environment and network variability** (DNS delays, staging restarts, CI runner contention) introduces non-determinism that test logic alone cannot eliminate.

Measuring flakiness is the prerequisite for controlling it. A practical approach is to track, per test, the fraction of runs that produce a result inconsistent with the majority over a fixed observation window. A rate below 2% means nearly every failure signals a genuine defect and can be acted on without manual triage. Without a measured baseline, failures remain ambiguous, and teams default to dismissal rather than investigation. Establishing that baseline is one of the explicit objectives of this project, and the measurement methodology is detailed in Chapter 6.

2.2.3 Test Independence and the Coupling Anti-Pattern

A test is independent when it produces the same result regardless of the order in which it runs, the subset it belongs to, or the state of the environment before it starts. Achieving this requires two things: each test sets up all the data it needs before executing its assertions, and it cleans up any side effects before finishing.

The coupling anti-pattern occurs when these requirements are not met. Coupled tests share state: they rely on records created by earlier tests, assume a specific platform configuration, or leave data behind that later tests consume. Meszaros documents this as the *Interacting Tests* smell in his catalogue of xUnit test anti-patterns [5]. Coupling has two concrete consequences. It prevents parallel execution: if test B depends on data produced by test A, both must run sequentially and in order. It also causes failure cascades: a failure in test A, even a flaky one, breaks test B regardless of whether the feature B validates is itself functional.

Both consequences appear among the documented limitations of the RF suite. The smoke suite's test-independence requirement (FR-03) addresses them directly.

2.2.4 Test Data Management Strategies

Three strategies exist for supplying a test with the data it needs to execute. Table 2.3 summarises their characteristics across the criteria most relevant to this project.

Strategy	Mechanism	Independence	Speed impact
Static fixtures	Pre-seeded records assumed present before any test runs.	Low: any modification to the shared state breaks dependent tests.	None at runtime, but fragile after environment resets.
API-driven setup	Data created via direct API calls before the test; deleted via API after.	High: each test owns its data entirely.	Low: API calls are fast, with minimal impact on total execution time.
UI-driven setup	Data created by navigating the application itself before assertions begin.	High: each test creates its own data via UI interactions, with no shared state across tests.	High: UI interactions are slower and add measurable time per test.

Table 2.3: Test data management strategies

Static fixtures create the coupling described earlier and conflict with the test-independence requirement. API-driven setup offers the cleanest path to full test independence, but it requires stable API access for each universe. UI-driven setup removes the API dependency, though at the cost of execution time, which works against the 10-minute target. For the LBC smoke suite, the right strategy varies by universe and journey type.

2.3 Modern DevOps Practices and Test Automation

2.3.1 Shift-Left Testing and the Fail-Fast Principle

Shift-left testing is the practice of moving validation earlier in the development lifecycle, closer to the point where a defect is introduced. The underlying argument is economic. A defect found before a commit costs minutes to resolve. The same defect found in production costs significantly more in investigation time, support overhead, and user trust. The practice is one of the foundational tenets of continuous delivery [6] and of the broader DevOps movement [7].

The fail-fast principle is complementary. A pipeline should surface a failure as quickly as possible and stop further processing once it does. Running expensive downstream stages against a build that is already broken wastes CI resources and delays the signal a developer needs to act. A multi-hour pipeline that reports a broken critical path only at

the very end is neither shift-left nor fail-fast.

Together, these two principles define a concrete requirement: the fastest possible verification of the most critical paths must run first. The smoke-gate pattern is the direct realisation of that requirement, enforced in practice through automated quality gates.

2.3.2 Automated Quality Gates in CI/CD Pipelines

A quality gate is a pipeline stage whose failure prevents downstream stages from executing. Gates are the operational realisation of continuous-integration discipline (Fowler [8]): a stage passes or fails, and the pipeline either continues or stops. The position of a gate determines its impact: a gate placed before a long regression suite saves the full runtime of that suite when it catches a critical failure early.

Two pipeline contexts are relevant here. On merge-request pipelines the goal is fast inline feedback during code review, and a lightweight lint gate is enough to deliver it; running a full smoke suite on every Merge Request (MR) would inflate cycle time without changing the review outcome. On deployment-bound pipelines, the risk profile changes: the artifact is about to advance, so a heavier gate that validates critical paths is justified. Gate granularity is therefore matched to the trigger event, not applied globally.

One design constraint follows from this. A smoke suite that takes 25 minutes is not an effective gate; it will be bypassed or ignored. The 10-minute execution target is not an arbitrary performance goal. It is the threshold below which the gate remains usable on every manual or scheduled trigger without becoming a bottleneck in its own right.

2.4 Browser Automation Frameworks

2.4.1 Selenium / WebDriver

Selenium is the foundational browser automation tool in the industry, and the WebDriver protocol it standardises is implemented by all major browsers. Figure 2.2 illustrates its architecture.

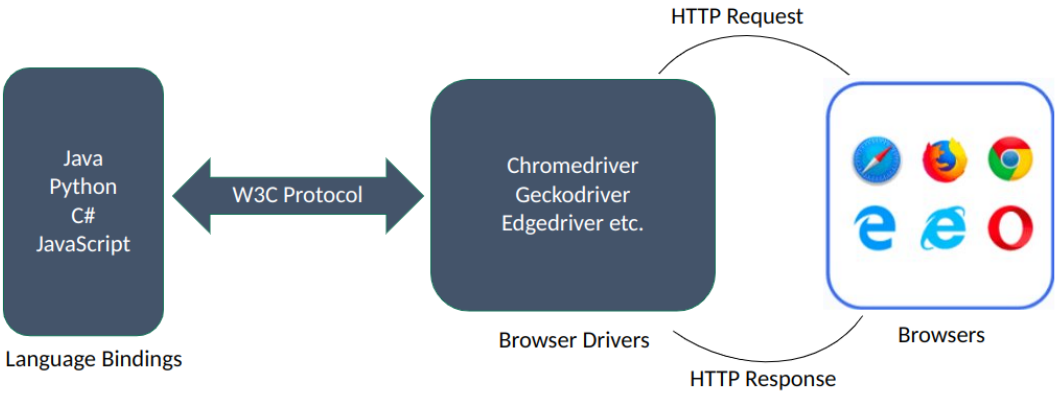


Figure 2.2: Selenium / WebDriver architecture [9]

Test code sends each command over HTTP to a driver process (ChromeDriver, GeckoDriver) that relays it to the browser. That per-command round-trip is the structural source of the timing flakiness described earlier.

2.4.2 Cypress

Cypress takes the opposite approach. Rather than driving the browser from an external process, it runs inside the browser's event loop. Test code in Node.js reaches the browser through a proxy server and a persistent WebSocket. This gives synchronous Document Object Model (DOM) access and browser-level network interception. That design removes most of the timing uncertainty WebDriver tools inherit from their HTTP layer. Figure 2.3 shows the architecture.

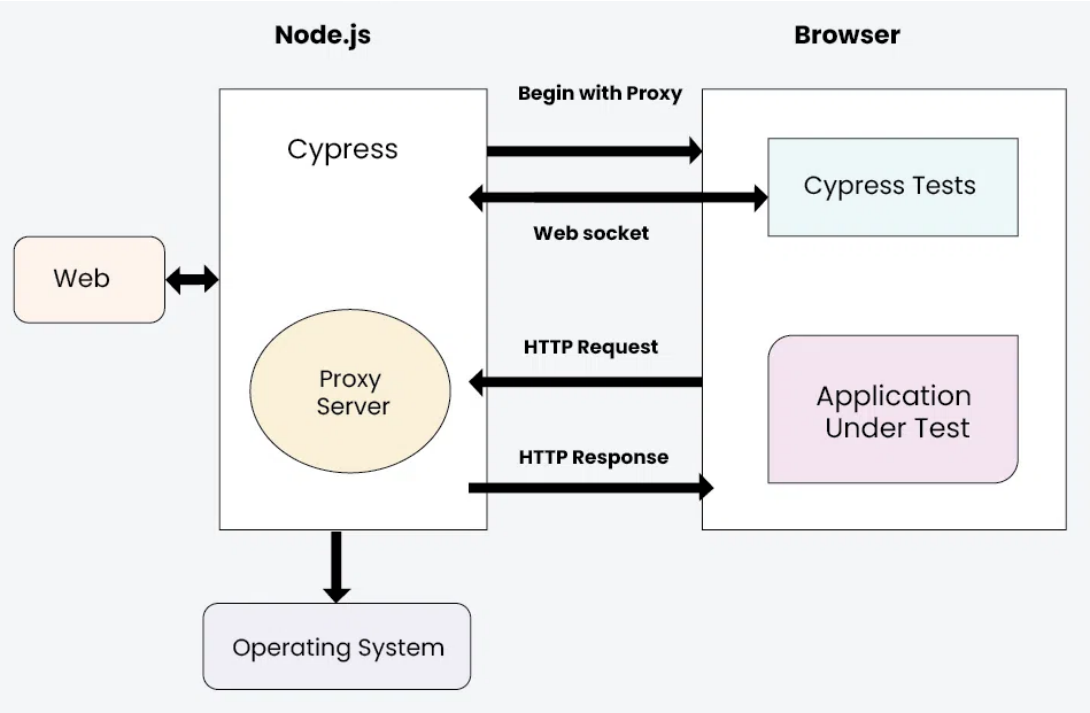


Figure 2.3: Cypress architecture [10]

Its limit is structural, though: a Cypress test runs in one browser context and cannot drive a Client and a Supplier session in parallel, so it cannot express LBC's cross-universe flows. We ruled it out on that basis. (`cy.origin()`, generally available since Cypress 12, covers cross-origin navigation, not parallel contexts.)

2.4.3 Robot Framework

Robot Framework (RF) [11] is a keyword-driven framework: tests use a tabular, readable syntax, a Python runtime executes them, and SeleniumLibrary delegates browser control to Selenium over WebDriver. Figure 2.4 shows the layered architecture.

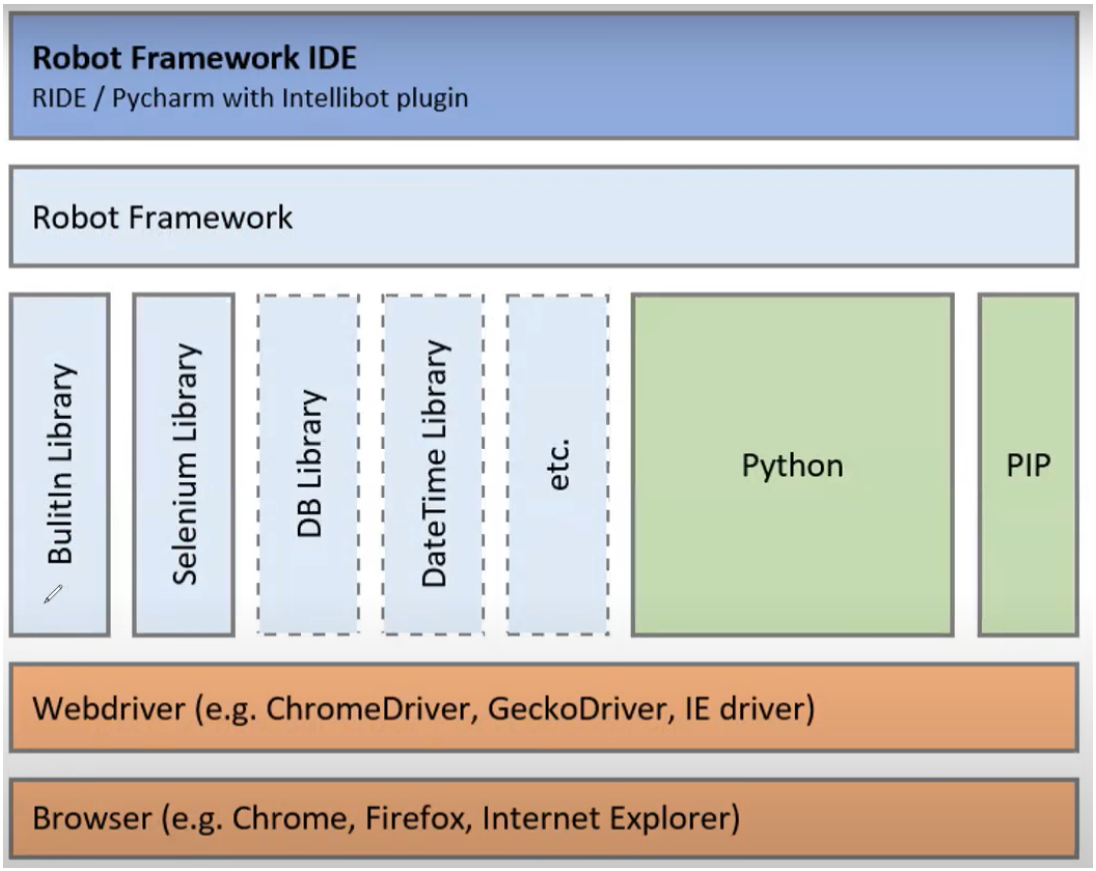


Figure 2.4: Robot Framework architecture [11]

RF is the team's existing tool. A test in tabular syntax is readable even by members without a programming background, and its keyword library covers most interactions without custom code, which makes RF a good fit for the broad regression coverage it already provides at LBC. With SeleniumLibrary, though, it inherits Selenium's WebDriver flakiness. The Python-to-SeleniumLibrary-to-WebDriver chain also blurs diagnostics: a failure's stack trace crosses several layers before reaching the browser, so the root cause is harder to localise. These limits belong to the binding, not to RF. A different binding, examined next, keeps the authoring model while replacing the execution layer.

2.4.4 Playwright

Playwright, from Microsoft [12], keeps one persistent, bidirectional connection to the browser instead of a separate HTTP command per action. It drives Chromium through the Chrome DevTools Protocol (CDP), and Firefox and WebKit through patched builds that expose an equivalent protocol. Figure 2.5 shows this architecture.

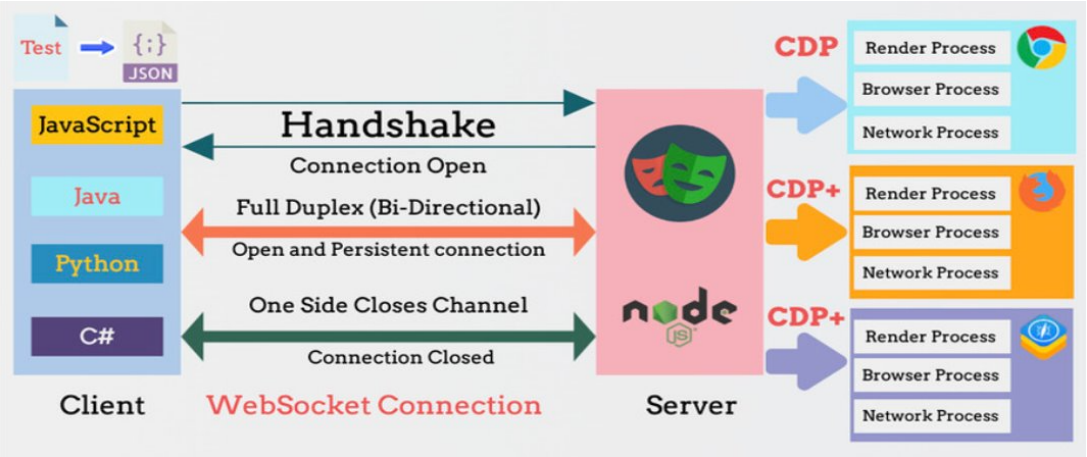


Figure 2.5: Playwright architecture [12]

CDP is the protocol that browser developer tools use: it streams network activity, the JavaScript execution context, and browser events to the runner over a WebSocket. Because that channel is persistent and event-driven, no HTTP round-trip sits between an action and its acknowledgment, which removes the main structural source of WebDriver flakiness. Playwright also brings three capabilities the LBC problem needs. It auto-waits on element state, which removes fixed waits. Native video and HAR capture close the Chapter 1 diagnostic gap. And multiple browser contexts in one process support the cross-universe flows. Its native authoring language, however, is TypeScript or JavaScript, which the next subsection addresses.

2.4.5 Robot Framework with Playwright

Browser Library (`robotframework-browser`) [13] closes that gap. It exposes Playwright through the keyword-driven model RF teams already use: tests stay in tabular RF syntax, but each keyword call now crosses a google Remote Procedure Call (gRPC) bridge to a Node.js process running Playwright, which drives the browser via CDP. The `Open Browser` keyword that once routed through Selenium and WebDriver now routes through Browser Library and Playwright, while page object resources, custom keywords, and the tabular syntax remain unchanged.

This binding matters where Robot Framework is already established. Migrating to standalone Playwright with TypeScript (TS) would force the team to maintain two suites in two languages. The keyword library built over years of regression work would not transfer. Browser Library keeps the smoke suite and the regression suite on one language and one set of conventions. The trade-off is a less mature binding than SeleniumLibrary, with different keyword and locator conventions and a smaller extension ecosystem. For a focused smoke suite that is acceptable; for a full regression migration it must be weighed carefully (deliverable D10).

2.4.6 Comparative Analysis and Technology Choice

Table 2.4 compares the five candidate stacks across the criteria that govern the smoke suite's architecture; the chosen stack is highlighted. In the table, DSL stands for Domain-Specific Language and JS for JavaScript.

Criterion	Selenium	Cypress	RF with Selenium-Library	Playwrigh	RF with Browser Library
Browser protocol	WebDriver (HTTP)	In-browser (event loop)	WebDriver (HTTP)	CDP (Web-Socket)	CDP (Web-Socket)
Multi-tab support	Yes	No	Yes (fragile)	Yes	Yes
Cross-origin frame support	Limited	Via `cy.origin()`	Limited	Yes	Yes
Native video capture	No	Yes	No	Yes	Yes
Native HAR trace capture	No	No	No	Yes	Yes
Auto-wait on element state	No	Partial	No	Yes	Yes
Authoring model	Multiple	TS / JS	RF keyword DSL	TS / JS	RF keyword DSL
Homogeneity with existing LBC suite	N/A	No	Full	No	Full

Table 2.4: Browser automation framework comparison

Four constraints decide the choice. *Parallel multi-context execution* rules out Cypress, which runs one browser context at a time. *CDP-based execution* rules out Selenium and RF with SeleniumLibrary, whose WebDriver HTTP layer is the structural flakiness source. *Native video and HAR* leave only Playwright and RF with Browser Library. *Homogeneity* with the existing RF suite then decides between those two in favour of RF with Browser Library, which keeps the team's keyword syntax. It is the only stack that satisfies all four, so we chose it.

Conclusion

Three things came out of this chapter. Smoke testing is a selective E2E layer that precedes regression, not a replacement for it. Flakiness, coupling, and poor diagnostics are identifiable problems with known solutions: CDP-based automation, full test independence,

and native trace capture. And among the five stacks evaluated, RF with the Browser Library is the only one that satisfies all four hard constraints at once. The next step is to take those conclusions and turn them into a concrete specification and architecture for the LBC platform, which is what Chapter 3 does.

Chapter 3

Requirements Analysis and System Design

Summary

3.1 System Requirements
3.2 Critical Path Analysis
3.3 System Modeling and Design

Introduction

Chapter 1 described the problem; this chapter defines the solution. We state the requirements, identify the critical user journeys that set the smoke suite's scope, and design the layered architecture and pipeline shapes that implement them. Every implementation decision in Chapter 4 traces back to a requirement or a constraint established here.

3.1 System Requirements

The system specified in this chapter is the smoke-test framework and its CI quality gate, not the LBC platform it exercises. We follow the canonical requirements-engineering decomposition (Sommerville [14]): identify the actors who interact with the system, then state the functional and non-functional requirements that bind their interaction.

3.1.1 Actors Identification

Two human actors operate the framework:

- **QA Engineer** (primary). Authors tests, defines page object resources, configures environments, runs the suite, and investigates failures.
- **Developer** (secondary). Triggers the suite indirectly through merge requests and consults its results on pipeline failures.

The CI pipeline is not an actor (it is the runtime environment hosting the framework's execution), and the platform's user universes are not framework actors either: they are the roles the framework's tests *simulate*, not parties that interact with the framework.

3.1.2 Functional Requirements

A functional requirement (FR) specifies what the system must do, as opposed to under what constraints it must do it. The nine requirements below derive from the critical path analysis and the project brief and are listed by descending priority in Table 3.1.

ID	Requirement	Priority
FR-01	Automate smoke tests for the Client and Supplier universes.	P1
FR-02	Automate smoke tests for the Admin universe and the two Freelancer journeys hosted in Supplier.	P1
FR-03	Each test must be fully independent: it owns its data creation and cleanup, with no shared state across tests.	P1
FR-04	Handle authentication and session management for all four user roles.	P1
FR-05	Support multiple test environments via configuration only.	P1
FR-06	Integrate the smoke suite as a dedicated CI/CD pipeline stage within the existing Robot Framework pipeline.	P1
FR-07	Generate a test results report after each pipeline run.	P2
FR-08	Publish the suite's evaluation metrics to a dashboard tracked over time.	P2
FR-09	Notify the team of each run's outcome and monitor pipeline health.	P2

Table 3.1: Functional requirements

3.1.3 Non-Functional Requirements

A non-functional requirement (NFR) specifies under which constraints the system must operate, rather than what behaviour it must produce. Table 3.2 lists the five non-functional requirements, each with a measurable target used to evaluate the suite in Chapter 6.

ID	Attribute	Requirement	Target
NFR-01	Performance	Smoke suite completes within a fast feedback window.	$<$ 10 minutes
NFR-02	Reliability	Low rate of failures unrelated to genuine defects.	$<$ 2% flakiness
NFR-03	Maintainability	Tests structured around the Page Object Model.	Code review validation
NFR-04	Scalability	Adding a new test or universe requires no framework-level changes.	Architecture review
NFR-05	Portability	Switching environments is a configuration change.	Config-only switch

Table 3.2: Non-functional requirements

3.1.4 Risk Register

The following risks were identified at the start of the project and informed both the architectural decisions and the implementation choices documented in Chapters 4 and 5. Table 3.3 lists each risk, its assessed likelihood and impact, and the mitigation strategy built into the framework.

ID	Risk	Likelihood	Impact	Mitigation
R-01	Auth0 silent-auth API changes invalidate the fast-path login mechanism.	Medium	High	Built-in fallback to the UI path; any regression surfaces immediately in the Teams notification.
R-02	Platform UI refactoring breaks CSS locators across multiple page resources.	Medium	High	Locators are isolated in named variables inside each page object; an update touches one resource at a time and is caught by CI linting before merge.
R-03	Shared pre-production fixtures modified or deleted by another team between runs.	Low	Medium	Read-only tests depend only on stable platform accounts; write tests own their records end-to-end.
R-04	GitLab runner load pushes wall-clock time over the 10-minute budget.	Low	Medium	Native auto-wait eliminates fixed sleeps; the 10-minute budget allows four minutes of headroom over the ∼6 min sequential baseline.
R-05	Suite maintenance abandoned after the internship ends.	Medium	High	Page-object model and Robocop linting enforce structural conventions automatically; an internal conventions file documents them; the migration roadmap (D10) provides a handoff plan.

Table 3.3: Project risk register

3.2 Critical Path Analysis

The smoke suite's scope is fixed by which user journeys earn a place in it. We set the selection criteria first, then catalogue the chosen journeys per domain.

3.2.1 Selection Criteria for the Smoke Suite

A journey qualifies for inclusion on two joint criteria: **business impact** (failure breaks the experience for a significant share of users) and **failure visibility** (the journey integrates authentication, navigation, data loading, and rendering, so failures would otherwise escape lower-level tests). Every smoke test is priority-one; a journey that does not warrant blocking the pipeline does not warrant a smoke test.

Two structural decisions follow. Authentication, although technically a precondition for every other test, is also the most heavily exercised path and earns its own *Auth* domain.

The Freelancer universe warrants a specific decision. At the account-model level it is a subtype of Supplier, sharing the same Auth0 tenant and session state. A separate freelancer pack would have duplicated the import surface, browser lifecycle, and authentication layer for no parallel-execution gain. The two Freelancer journeys therefore sit inside the Supplier domain under a *Supplier Freelancer* test-name prefix, keeping the actor explicit in reports without a redundant directory.

The four-universe view of the platform thus maps onto four test domains: Auth, Admin, Client, Supplier. Deliverable D1 grades the catalogued journeys as P1, P2, or P3; the smoke gate covers the P1 set, with the full catalogue (including the deferred entries discussed in the Perspectives section) reproduced in Appendix A.

3.2.2 Critical User Journeys per Domain

Every domain except Auth maps one test to one priority-one journey; Auth's three journeys (admin login, user login, log-out) bundle into a single end-to-end test so they share one Auth0 session. The suite therefore consists of 18 tests automating 20 priority-one critical user journeys (CUJs) across four domains, summarised in Table 3.4. In the table, RFP stands for Request for Proposal, BDC for Bon de Commande (purchase order), and CRA for Compte Rendu d'Activité (activity report). The per-domain design class diagrams in Appendix B list each test under its parent resource, giving the journey-to-test mapping inline with the structural model.

Domain	Tests	CUJs	Coverage focus
Auth	1	3	Auth0 redirect flow, session establishment, log-out path (three CUJs bundled in one test).
Admin	4	4	Back-office configuration and oversight: audit log, email management, administrative purchase orders, region configuration.
Client	6	6	Core business cycle on the requester side: RFP browse, draft creation, BDC / CRA list inspection, proposals view, and the partner-domain access check.
Supplier	7	7	Supplier-side mirror flows (marketplace search, applications, BDC, CRA, invoices) plus the two Freelancer journeys (available missions, skill search on profile).
Total	**18**	**20**	

Table 3.4: Smoke suite test count and CUJ coverage per domain

3.3 System Modeling and Design

With the scope fixed, we model the framework that delivers it, from its external behaviour down to the pipeline that runs it.

3.3.1 Use Case Diagram

The external behaviour of the framework, as seen by its two actors, is captured by the use case diagram in Figure 3.1. Use cases are grouped into four packages corresponding to the operational concerns of a test framework: authoring, execution, diagnostics, and integration.

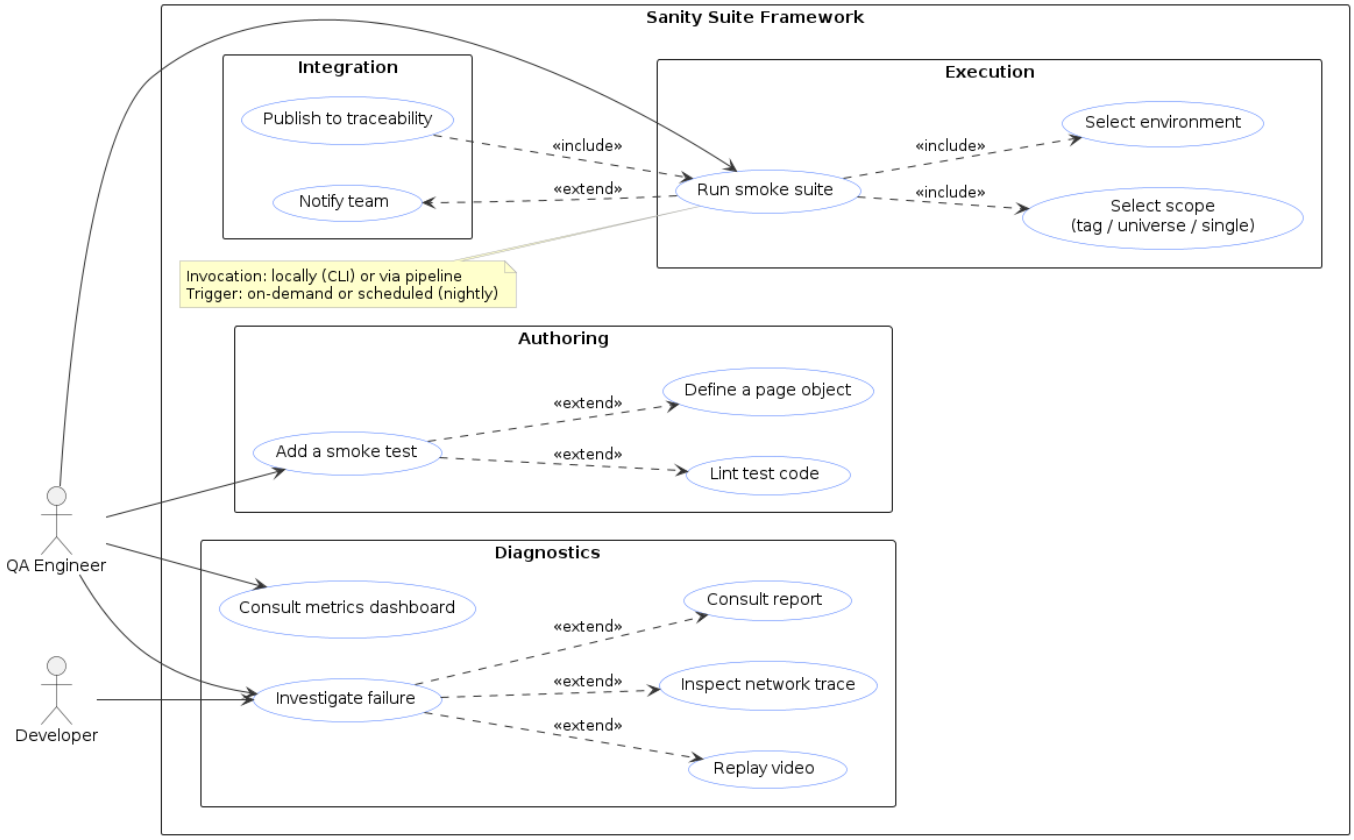


Figure 3.1: Use case diagram of the framework

The QA Engineer authors tests, runs the suite, consults the metrics dashboard, and investigates failures; the Developer investigates failures on a red pipeline, consulting the report, the video, and the network trace. Two integration use cases fire automatically from *Run smoke suite*: every run includes *Publish to traceability*, and a completed run may extend into *Notify team*. The note attached to *Run smoke suite* captures two operational dimensions Unified Modeling Language (UML) does not encode as separate use cases. The suite can be invoked locally or through the pipeline, and pipeline runs can be on-demand or scheduled. Behaviour is identical across all four combinations.

3.3.2 Sequence Diagram: Running the Smoke Suite

The use case diagram fixes what each actor can do; the sequence diagram in Figure 3.2 shows how the central *Run smoke suite* use case unfolds over time, from the QA Engineer's trigger through to the dashboard and the team notification.

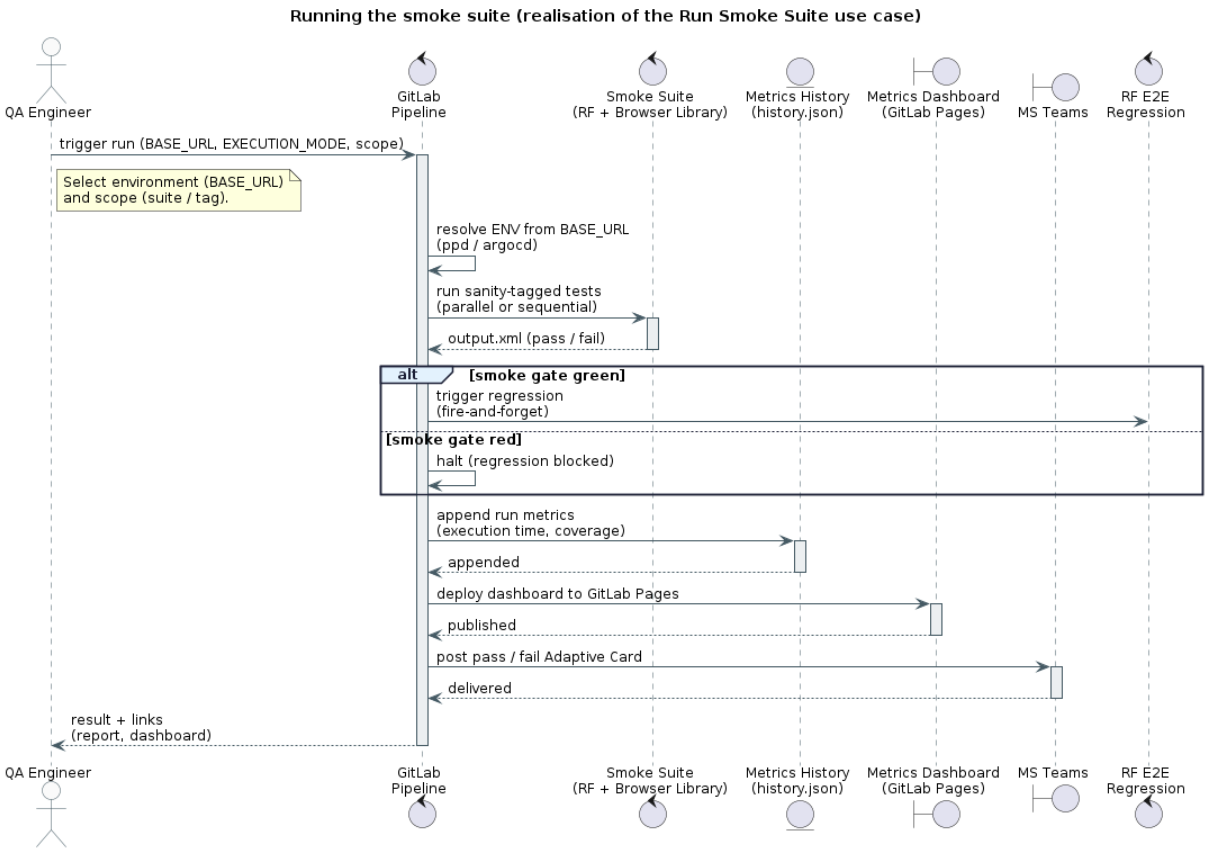


Figure 3.2: Sequence diagram: running the smoke suite

The QA Engineer starts a run by choosing an environment (the `BASE_URL`) and a scope (a suite or tag). The pipeline resolves the target environment from the URL, then runs the sanity-tagged tests in parallel or sequentially. Once the suite returns its verdict, the pipeline evaluates the smoke gate immediately: a green result dispatches the downstream regression suite as a fire-and-forget call; a red result halts and the regression suite never runs. Regardless of the gate outcome, the pipeline then appends the run metrics to the history file, redeploys the dashboard on GitLab Pages, and posts a pass-or-fail Adaptive Card to Microsoft Teams before returning the result links to the QA Engineer.

The diagram stays at the operational level, where the lifelines are the QA Engineer and the subsystems a run drives. The object-level mechanics beneath a single keyword, the page-object-to-utility delegation and the Browser Library binding, are described with the implementation in Chapter 4.

3.3.3 Layered Architecture

The framework decomposes into four horizontal layers with a one-way dependency flow from top to bottom, as shown in Figure 3.3.

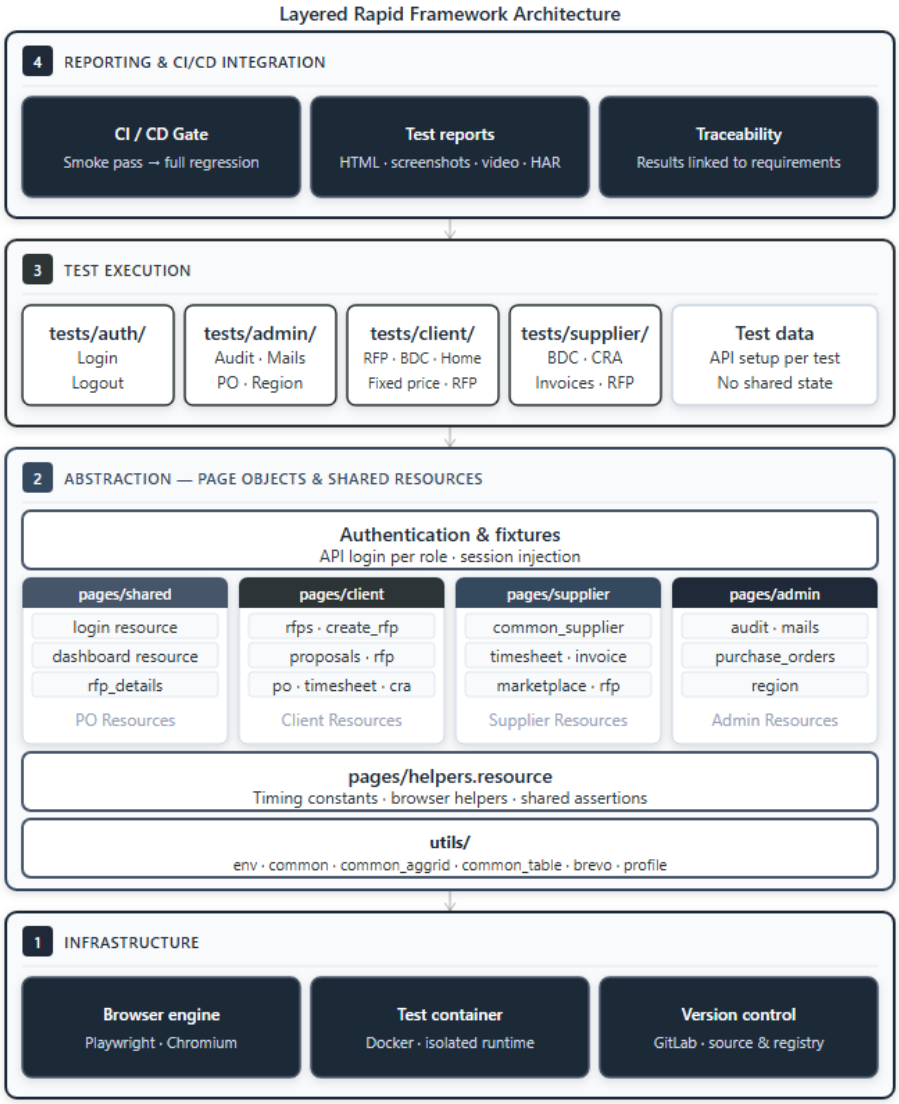


Figure 3.3: Layered architecture of the framework

- **Layer 1: Infrastructure.** Chromium driven by Playwright through Browser Library, a Docker test container pinning the runtime dependencies, and GitLab as the version-control and CI host.

- **Layer 2: Abstraction.** Page-object resources organised by universe (`pages/admin`, `pages/client`, `pages/supplier`, `pages/shared`), each universe exposing one aggregator resource as its import surface. A shared helpers resource consolidates timing constants, browser-session helpers, and assertions; authentication and configuration are resolved here from environment variables.

- **Layer 3: Test Execution.** Test packs (one per domain) consume the abstraction layer, with test data created per test rather than shared. The directory structure mirrors Layer 2 one-to-one, which is what makes the package-by-feature pattern visible in the file system.

- **Layer 4: Reporting and CI/CD Integration.** The CI/CD gate implements the smoke-first behaviour. Reports include HTML output, screenshots, video recordings, and HAR traces; the traceability layer routes them to a Microsoft Teams channel and feeds the metrics dashboard published on GitLab Pages.

Two *architectural patterns* govern the structure. The **layered architecture** pattern (Fowler [15]; Bass, Clements, Kazman [16]) groups responsibilities into horizontal layers and enforces a one-way dependency rule between them: each layer depends only on the layers beneath it. The framework applies this rule across its four layers, so tests depend on page objects, page objects depend on helpers and utilities, and no arrow points upward. The **package-by-feature** pattern [17] groups source files by the business feature they serve rather than by their technical type (controller, model, view). Each feature then forms a vertical slice that owns all its files. We apply it to Layers 2 and 3 by partitioning `pages/` and `tests/` by universe rather than by file type: each universe is its own slice that mirrors the same shape on both layers.

3.3.4 Package Dependency Diagram

The layered view fixes the dependency direction; the package dependency diagram fills in the packages that populate each layer. The full diagram is reproduced in Figure 3.4. The figure stays horizontal so that every package and association remains legible at full size. The page that carries it stays portrait, so the document prints without any rotated paper.

The diagram is organised into four packages that mirror the four layers: *tests* (one pack per domain plus a shared test-data module), *domain pages* (the AdminModule, ClientModule, SupplierModule), *shared pages* (LoginPage, DashboardPage, Helpers), and *utils* (BrowserSession, the grid and table utilities, the environment resource). The dependency arrows confirm the one-way layered rule established above: every dependency points downward, never up.

Two design patterns govern object-level collaboration. The **Page Object Model** (POM) [18] represents every page or significant UI component as a dedicated class. Each class encapsulates its element selectors and interaction methods, so the test calls business-level methods rather than querying the DOM directly. We realise it as one resource per page: tests read *Open RFP Detail* instead of the raw locator chain, and a UI change touches the corresponding resource alone. The **Facade** pattern [19] provides a unified entry point to a set of related interfaces, so the caller imports one surface and gains access to all of them. Each domain exposes one aggregator (`client`, `supplier`, `admin`) that re-exports every page resource under its directory, so a test imports one resource per universe.

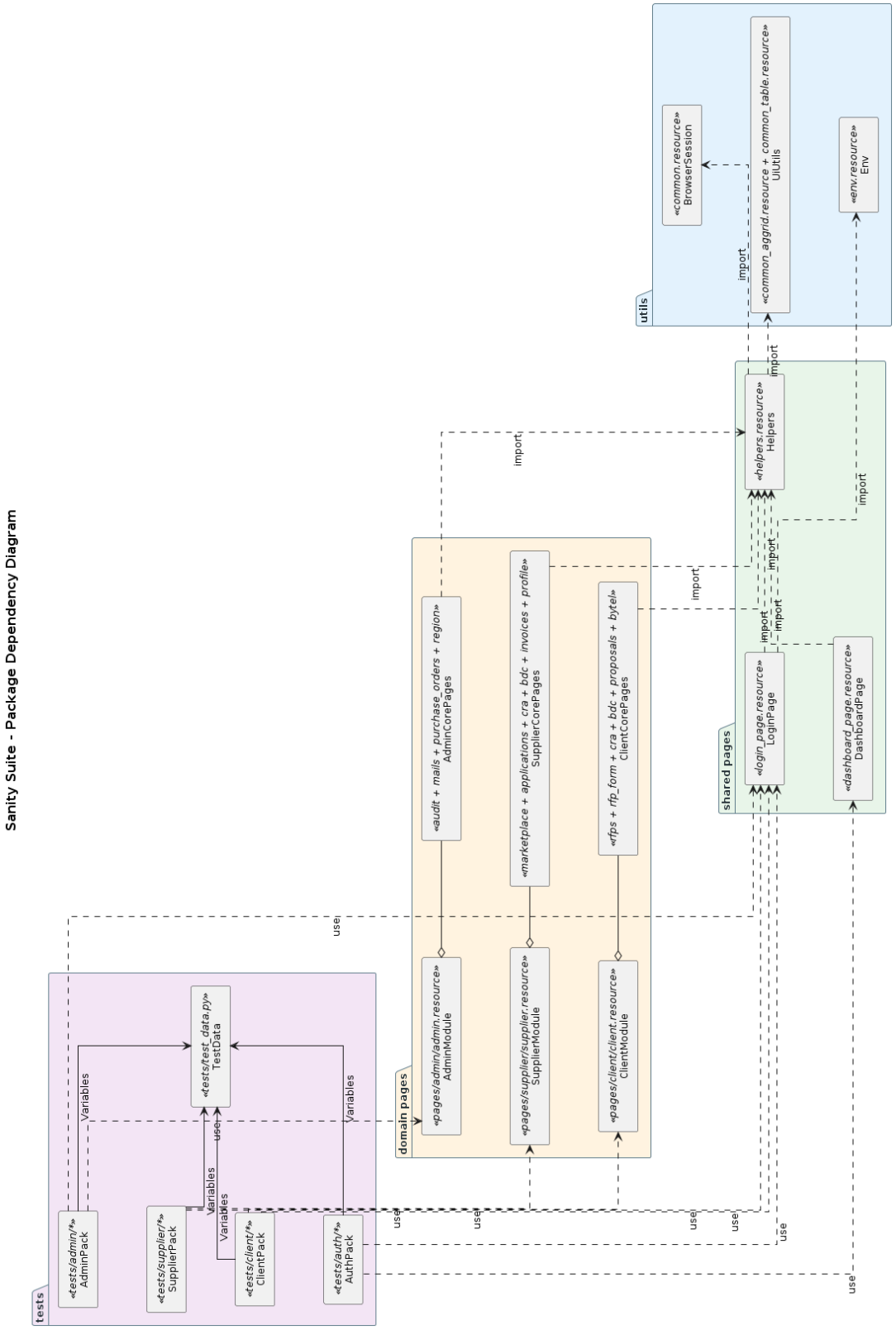
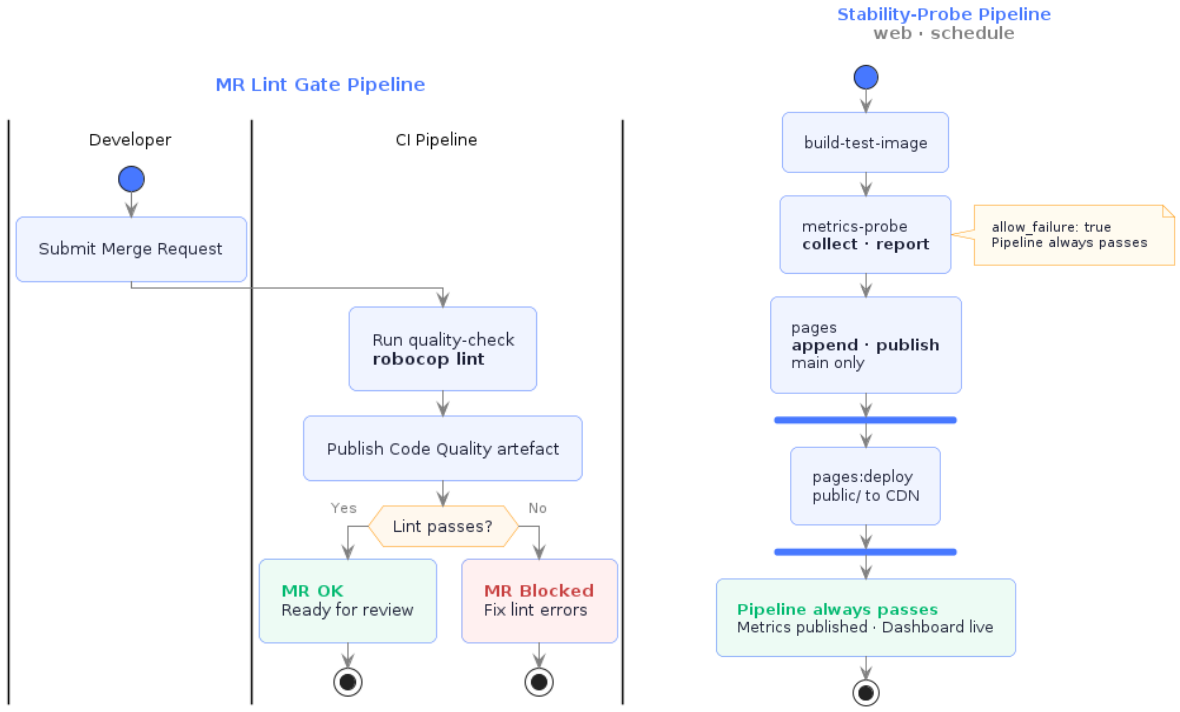


Figure 3.4: Package Dependency diagram of the framework

3.3.5 Pipeline Design

The framework's runtime architecture is matched by a deployment-side architecture: the shape of the CI/CD pipeline that runs it. Three flows cover every event the project reacts to. A lightweight merge-request gate runs static linting only (Figure 3.5(a)). A full smoke-plus-trigger pipeline serves manual and scheduled launches (Figure 3.6). And a dedicated stability probe pipeline bypasses the smoke gate to feed the metrics dashboard (Figure 3.5(b)). The deployment-time realisation of each shape on GitLab is covered in Chapter 5.



(a) Merge-request gate: lint only.

(b) Stability-probe pipeline: `notify` omitted; the probe runs with `allow_failure: true`.

Figure 3.5: Auxiliary pipeline activity diagrams: merge-request gate and stability probe

The full smoke pipeline starts by building the test image, then branches on the `EXECUTION_MODE` variable: parallel mode dispatches four domain jobs via pabot, while sequential mode runs a single robot process. Both paths converge at the `pages` stage, which appends run metrics and redeploys the dashboard, followed by the Teams notification. A single gate, *Smoke Pass?*, determines the final outcome: a green run dispatches the downstream regression suite as a fire-and-forget call and exits with Pipeline PASS; a red run exits with Pipeline FAIL and the E2E trigger never fires.

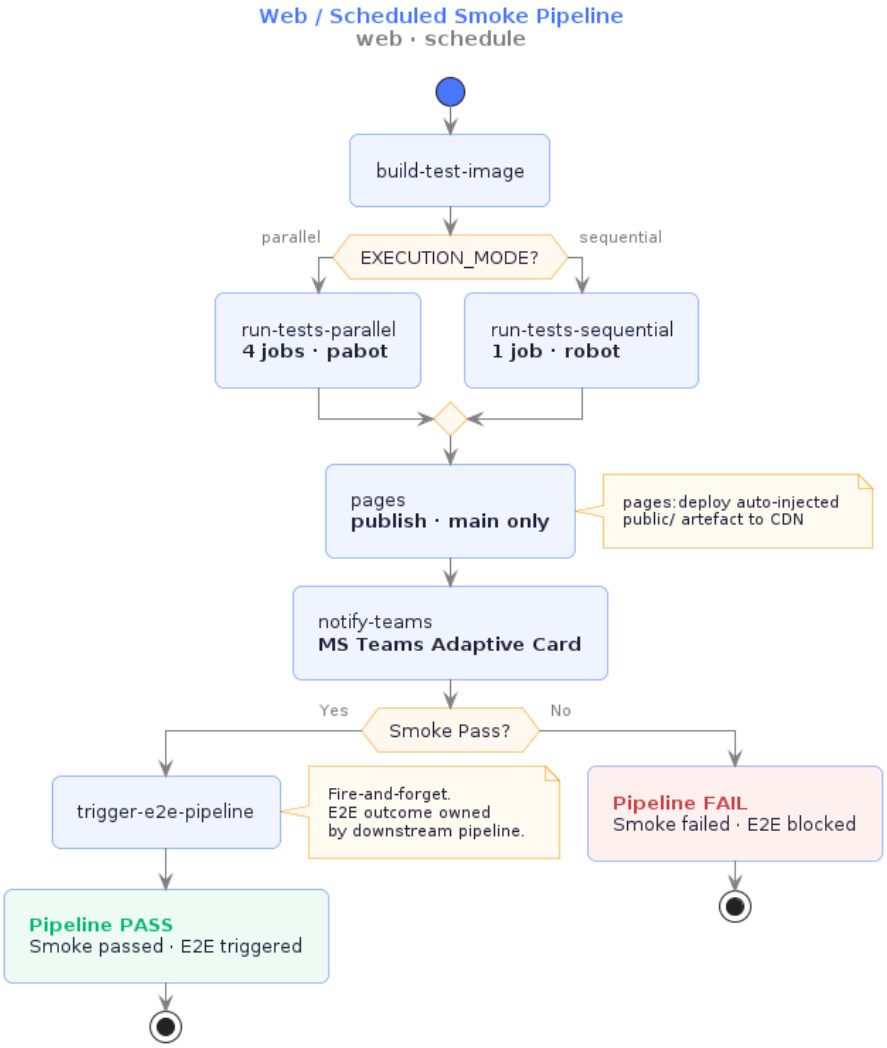


Figure 3.6: Web and scheduled pipeline activity diagram

Conclusion

The system specification and the architectural design are now complete. The actors are identified, the functional and non-functional requirements are formalised with measurable targets, and the risks that informed the design are listed alongside the mitigations baked into the framework. The critical path analysis fixes the scope of the smoke gate at twenty priority-one journeys spread across four test domains. The framework architecture splits the codebase into four layers under a one-way dependency rule. Two architectural patterns (layered architecture, package-by-feature) and two design patterns (POM, Facade) govern the collaboration between them. Three pipeline shapes close the deployment loop. Chapter 4 covers the implementation of each decision in practice, domain by domain.

Chapter 4

Implementation of the Smoke Test Framework

Summary

4.1 Development Environment and Tooling
4.2 Framework Skeleton
4.3 Authentication and Session Management
4.4 Test Data Strategy
4.5 Smoke Test Implementation per Domain
4.6 Code Quality and Linting
4.7 Metrics Dashboard

Introduction

Chapter 3 defined what to build; this chapter covers how it was built. The sections follow the order in which the framework came together. The toolchain and the source-file skeleton come first, because nothing runs without them. Authentication and the test-data strategy come next, since every test depends on both. The per-domain packs follow, and the chapter ends with the two pieces that keep the suite maintainable as it grows: the linting layer and the metrics dashboard.

4.1 Development Environment and Tooling

The toolchain is deliberately small. Each tool has one job in the loop of writing, running, or checking tests, and the suite needs nothing beyond this list to run from a clean checkout. Table 4.1 gives each tool and its role.

	Tool	Role in the framework
	**Visual Studio Code**	Authoring environment with keyword completion, go-to-definition, and on-save linting via the RF Language Server.
	**Python 3.10 or later**	Hosts the Robot Framework runtime and the helper scripts under `traceability/` that parse `output.xml`.
	**Robot Framework**	Test orchestration: keyword resolution, suite lifecycle, reporting.
	**Browser Library**	Robot Framework binding to Playwright. Exposes Playwright's capabilities through the keyword-driven syntax.
	**Playwright**	Browser engine that drives Chromium over the Chrome DevTools Protocol (CDP); pulled in by `rfbrowser init`.
	**Chromium**	Headless browser driven by Playwright; pinned to a specific build inside the Docker image for reproducible execution.
	**pabot**	Parallel runner used in the CI parallel mode; one process per domain.
	**Robocop**	Static linter for Robot Framework source files; configured through `robocop.toml`.
	**Docker**	Reproducible execution container, published to the GitLab project registry.
	**Git**	Source control; commit history and branches drive both local development and CI triggers.
	**GitLab**	CI host and registry. Pipelines, schedules, Container Registry, and Pages are all served from the same project.
	**Jira**	Project tracking via a Kanban board; test case management and result traceability handled through the Xray plugin.
	**Microsoft Teams**	QA notification channel; receives an Adaptive Card after each pipeline run via the `notify-teams` stage.
	**HTML5**	Structure layer of the metrics dashboard: `dashboard/index.html` served via GitLab Pages.
	**CSS3**	Styling layer of the metrics dashboard: `dashboard/styles.css`.
	**JavaScript**	Logic layer of the metrics dashboard: `dashboard/app.js` reads `history.json` and renders the four metric cards through Chart.js.

Table 4.1: Development tooling and the role of each component

Local setup is two commands. `pip install -r requirements.txt` pins the Python dependencies, and `rfbrowser init` downloads the Chromium build Browser Library drives. After they finish, the suite runs against any configured environment.

Local execution mirrors the CI path through the same three modes: `robot tests/` (sequential), `pabot` with a distinct `-pabotlibport` per instance (parallel), and `PWDEBUG=1 robot tests/` for Playwright's visual debugger. The GitLab pipeline routes between the two production modes via the `EXECUTION_MODE` input (Chapter 5). A test that fails in the pipeline must therefore be reproducible on a developer's machine without any CI-specific scaffolding.

4.2 Framework Skeleton

4.2.1 Project Structure

The repository layout follows the architecture. Each layer and each supporting concern has its own directory at the root: infrastructure scripts, traceability and metrics tooling, the dashboard site, shared utilities, the universe page objects, and the test packs. The page-object tree and the test tree mirror each other one to one. That is the package-by-feature pattern in practice. Appendix C reproduces the full tree as it stood at the end of the implementation phase.

4.2.2 Shared Resources and Utilities

The utility layer holds every shared resource that page objects and tests depend on. Three responsibilities live there:

- **Library imports.** `Browser`, `Collections`, and `String` are declared in one resource and never redeclared, so Browser Library's runtime singleton stays in a single audited place.
- **Environment routing.** A resource dispatches on the `ENV` variable set at startup and populates the runtime globals (`BASE_URL`, `USERNAME`, `PASSWORD`, and the rest) from process environment variables, so no test holds a literal URL or credential.
- **Browser lifecycle.** Two keywords open and close a Chromium context with the configured viewport, recording settings, and Playwright traces; the shared test initialisation invokes them, so every test inherits the same lifecycle.

Two further utility resources hold the UI primitives that page objects delegate to. The first owns every AgGrid interaction: header retrieval, row filtering, sort, and cell assertions. The second does the same for the PrimeVue DataTable, used on the admin purchase-order

list and the Supplier applications list. Page objects delegate to these utilities by passing domain arguments (column identifier, expected text, row index); no grid or table selector ever appears inside the page-object tree. This is the rule that keeps a future grid-library migration localised to one page resource at a time.

4.2.3 Page Object Resources and Universe Aggregators

A central helpers resource exposes the four timing constants used across the suite (`TIMEOUT`, `SHORT_WAIT`, `NORMAL_WAIT`, `LONG_WAIT`) and a small set of generic browser helpers. It also re-exports the grid/table utilities and the login resource, so a page object imports this one resource and inherits the full toolbox.

Each universe directory follows the same convention: one aggregator resource named after the universe, plus one resource file per page or screen family. The aggregator is the Facade described in Chapter 3; a test imports the aggregator alone and the internal page split remains hidden behind it. When a new page is added under a universe, only the aggregator is touched and the test files are not.

The shared sub-tree holds the two cross-universe page objects: the login page and the dashboard page. Both are reachable through the helpers resource, so any page object can call them without redeclaring the import or risking a circular dependency.

4.3 Authentication and Session Management

Authentication cuts across every test in the suite. The 18 smoke tests each open a browser, log in, exercise a critical path, and tear down. If each one performs a full Auth0 redirect, the suite spends a significant fraction of its runtime just authenticating. The implementation therefore separates *logging in* into two paths: a UI path used exclusively by the login test itself, and an API fast path that every other test uses.

- **UI path (canonical).** The first login of the Robot process navigates to the platform, completes the Auth0 form, and waits for the home page. Three pieces of state make up the authenticated session and are captured at that point: the Auth0 Software Development Kit (SDK)'s localStorage entries, the backend session cookie, and the role-specific landing URL.

- **API fast path.** Once the session is cached, every subsequent login posts to the backend login endpoint with the API password, then replays the cached state (localStorage entries, Auth0 SDK cookies, session cookie) into the new browser context. A direct navigation triggers Auth0's silent-auth probe, which finds the injected state and skips the redirect. Each subsequent login is then far cheaper

than a full UI login, a saving Chapter 6 quantifies.

- **Cache scope.** Cached state lives in Robot Framework globals, so its scope is the Robot process. Each pabot worker pays one UI login on its first test and reuses the fast path for every subsequent test it runs (four parallel workers ⇒ four UI logins total). The login test itself always uses the UI path, since it exists to validate that flow.

- **Fallback.** When the API password is missing or the backend login endpoint errors, the keyword silently reverts to the UI path. This covers developer machines with absent secrets and environments where backend credentials have not yet been provisioned.

4.4 Test Data Strategy

The test-data layer has two jobs. Each test gets the fixtures it needs before it starts. No test may leave state behind that another could pick up. Together these keep every test independent of the others. Read-only and write tests achieve that differently.

4.4.1 Centralised Fixtures Module

A single Python module under `tests/` exposes every account, entity reference, and string fixture the suite consumes. It is imported by each test as a `Variables` declaration, so its top-level names appear in the Robot variable namespace as `${CLIENT_USERNAME}`, `${KNOWN_RFP_REFERENCE}`, and so on. Tests never embed literal strings: usernames, partner identifiers, RFP titles, and other fixture values all flow through this module.

The module also generates run-scoped identifiers for write paths. A test that creates a draft RFP requests a uniquely-prefixed title made of a stable prefix, a date stamp, and a short random suffix. This keeps two concurrent runs from colliding on the same draft title. It also keeps every created record attributable to the run that made it, even hours later.

4.4.2 Read-Only Tests

The majority of the smoke tests are read-only: they navigate to a list screen, exercise filter / sort / search controls, open a detail page, and verify rendered fields. These tests do not require fixtures of their own. They depend only on pre-existing platform records that are stable across runs: a Client account, a Supplier account, a partner enterprise tenant. These references are declared as constants in `test_data.py`.

The read-only assumption is enforced at code review: no read-only test calls a mutating API or performs a UI write. A mutating call inside a read-only test would couple it to its siblings.

4.4.3 Write Tests

A small number of smoke tests exercise write paths. The canonical example is *Client Can Create And Delete Draft RFP*, which creates a draft RFP and deletes it within the same browser session. The pattern is consistent: a write test takes ownership of the records it creates, performs its assertions, and removes those records before finishing. Cleanup happens in the test body rather than a teardown keyword, so a failure in the assertion phase still triggers the deletion step on the way out.

We chose UI-driven creation over API-driven creation for write smoke tests, despite the higher execution-time cost. The reasoning is specific to the smoke context: a write flow is a critical user journey precisely because the UI is what users interact with. Bypassing the UI to create the record would validate the backend without touching the path the smoke gate is meant to guard. For the regression suite, where coverage breadth matters more than fidelity to the user path, API setup remains the right strategy and is used extensively.

4.5 Smoke Test Implementation per Domain

Each domain contributes one test pack under `tests/`. The pack imports its universe aggregator and the shared `test_data` module, declares its tests, and inherits the suite lifecycle from `tests/__init__.robot`. The pack composition follows the same template across all four domains; the differences below concern scope and the specific page objects each pack consumes. Listing 4.1 shows a representative Client smoke test. Every step is a domain-named keyword that delegates to a page object under `pages/client/`; there are no inline locators and no raw browser calls.

Listing 4.1: Representative smoke test (Client domain, RFP list flow)

```
*** Settings ***
Documentation Client RFP smoke tests.
Resource ../../pages/client/client.resource
Resource ../../utils/common.resource
Variables ../test_data.py
Suite Setup Start Browser Session
Suite Teardown End Browser Session

*** Test Cases ***
Client Can Browse And Inspect RFP List
    [Tags] sanity client rfp list
    [Documentation] *Author: Ahmed Silini*
    ... Browse the RFP list, filter by reference, and open
    ... a detail page.
    Log As User USERNAME PASSWORD
    Navigate To RFP List
    Filter RFP List By Reference KNOWN_REF
    Open First RFP From List
    Verify RFP Detail Page Is Displayed
```

4.5.1 Auth Domain

The Auth pack holds one end-to-end test for the login flow itself, the path every other test takes for granted. It bundles three priority-one journeys into a single run: an administrator logs in, opens a user session from the admin panel, then logs out. The test then checks that the admin panel is restored, with user search available. This is the only test that logs in through the UI rather than the API fast path of Section 4.3, since its job is to validate that flow.

4.5.2 Client Domain

The Client pack implements the requester-side journeys of the contract cycle. Each test maps to a single critical user journey:

- *Browse and inspect the RFP list.* Navigate to the RFP list, apply sort and filter on a referenced or non-referenced entry, open the first matching item, and assert that the detail page loads.
- *Create and delete a draft RFP.* Fill the required fields (entity, title, location, duration, budget) with a run-scoped title, save the draft, assert it appears in the list, and delete it on the way out. This is the one write path in the pack.
- *Browse and inspect the BDC list.* Open the purchase order list, sort and filter, open one BDC, assert detail loads.
- *Browse and inspect the CRA list.* Open the activity report list, sort and filter,

open one CRA, assert detail loads.

- *View the Propositions list.* Open an RFP, navigate to its Propositions tab, assert the list loads.
- *Partner-domain access.* Hit a cross-tenant URL, retry up to three times to absorb transient DNS and certificate-provisioning delays on freshly-deployed environments, assert routing succeeds.

4.5.3 Supplier Domain

The Supplier pack implements the supplier-side journeys of the contract cycle. It also hosts the two Freelancer journeys, since Freelancer is treated as a Supplier subtype as discussed in Section 3.2. The applications list is rendered with a grid component distinct from AgGrid, so the table utility was extended to absorb it. That keeps a single source of truth (SSOT) for grid mechanics.

- *Search the RFP marketplace.* Access the marketplace, enter a search query, assert results display.
- *Browse and inspect the Application list.* Navigate to Applications, assert the list loads, open one application, assert detail loads.
- *Browse and inspect the CRA list.* Sort, filter, open one CRA, assert detail loads.
- *Browse and inspect the BDC list.* Sort, filter, open one BDC, assert detail loads.
- *Browse the Invoice list.* Navigate to Invoices, assert the list loads with the expected column headers.
- *Freelancer: browse available missions.* Navigate to the missions section, assert the list loads.
- *Freelancer: search a skill on profile.* Open the Freelancer profile, run a SkillsAlly skill query, assert the autocomplete results appear.

4.5.4 Admin Domain

The Admin pack implements the back-office journeys focused on configuration and oversight. The scope is entirely read-only by design, so the only fixture per test is an admin account and no cleanup step is required.

- *View the audit log.* Navigate to the Audit section, assert the log loads and contains entries.

- *View the mails interface.* Navigate to the Mails section, assert the interface loads.
- *View the purchase-order list.* Navigate to the admin Purchase Order list, assert the PrimeVue table loads.
- *View the region configuration.* Navigate to the region configuration page, assert the page loads with the expected regions.

One critical user journey from the requirements specification was reformulated during implementation: the data-export trigger initially documented in D1 fires an asynchronous backend job whose completion signal cannot be observed within the smoke time budget. We moved it to the regression suite and replaced it with a deterministic read on the admin purchase-order list, which exercises a comparable oversight surface with a stable signal.

4.6 Code Quality and Linting

The codebase is small today but will grow as later phases add tests. The linting layer codifies the conventions that keep it navigable as it grows, and the CI gate in Chapter 5 makes those conventions enforceable on every merge request.

Robocop is the static linter for Robot Framework, configured at the project root and committed to the repository so the same rules apply locally and in CI. A short set of authoring conventions, documented in an internal conventions file, governs every new test or page object. Three rules concentrate most of the value:

- Every keyword carries a `[Documentation]` block: the author, a concise description, the starting screen where relevant, and each argument and return value with its type. It also ends with a self-verifying assertion. Its `message=` states what the keyword expected. Robot Framework prints that line when a test fails on it, so the failure points straight at its cause.
- Locators are unique, written as CSS selectors or HTML `id` matches, and stored in named variables on the line before their first use. XPath and inline locators are banned.
- No page-object keyword touches a grid or table selector: those live in the AgGrid or DataTable utility, and the page object delegates to them.

The pages-versus-utils boundary is enforced here by Robocop's pattern rules and by code review.

Linting runs in three places. The RF Language Server flags violations in the editor as the developer types. `robocop check pages/ utils/ tests/` runs locally before a

commit. The CI `quality-check` stage runs the same command on every merge request and publishes the results as inline diff annotations (Chapter 5). Violations on lines a developer adds must be fixed before the merge request can land. Pre-existing violations in `main` are tracked in a backlog and cleared over time, which blocks new debt without forcing a big upfront cleanup.

4.7 Metrics Dashboard

The test suite ships with a static, LittleBig Connection-branded dashboard that tracks the four evaluation metrics over time. The dashboard is implemented as plain HTML, CSS, and JavaScript with no build step and is served via GitLab Pages.

4.7.1 Stack and Data Contract

The front-end consists of `dashboard/index.html`, `dashboard/styles.css`, and `dashboard/app.js`. There is no build step. The three files are served as-is by GitLab Pages, which keeps the dashboard a small, dependency-free directory in a project whose only runtime dependency is Python.

All four cards read from `dashboard/data/history.json`, a versioned, append-only JSON file (`schema_version: 1`). The file contains three arrays: `pipeline_runs` (one row per regular pipeline, written by `traceability/publish_metrics.py`), `stability_probes` (one row per probe run, written by `traceability/analyze_probe.py`), and `coverage` (refreshed on every `publish_metrics.py` invocation from the CUJ catalogue). Because no pipeline run ever overwrites earlier rows, the whole history is reconstructible from the file without a database. The versioned schema also lets new fields be added without breaking how older rows render.

4.7.2 Visual Identity and Theming

The dashboard adopts LittleBig Connection's visual identity rather than the Chart.js defaults, so it reads as part of the LBC tooling family and screenshots drop cleanly into Teams threads and internal reports. The brand palette, shown in Figure 4.1, is the one used on the LBC product surfaces: charcoal (`#1C2036`) for chrome, LB blue (`#4778FF`) for the parallel-run series, mint (`#00B96F`) for affirmative states, and muted red, orange, and slate for target lines, sequential runs, and per-test diagnostics. Every value is declared once as a CSS custom property on `:root`, so a future brand refresh touches one file rather than every chart.



Figure 4.1: LittleBig Connection dashboard palette

Typography inherits the platform's Inter sans-serif stack, and a light/dark toggle persists its choice in `localStorage`. The four cards sit in an equal two-by-two grid that fits one laptop viewport and collapses to a single column on smaller screens. The rendered dashboard, in both themes and with its per-card interaction controls, is reproduced in Chapter 6 alongside the results it visualises.

Conclusion

The suite is built: 18 smoke tests across four packs, an API fast path that cuts per-test login cost to roughly one second, and a dashboard that publishes the evaluation metrics on every pipeline run. What remains is making it operational. Chapter 5 describes how the suite is deployed as the first gate in LBC's CI/CD pipeline.

Chapter 5

Deployment and CI/CD Integration

Summary

5.1 Containerisation and Test Image
5.2 Pipeline Implementation
5.3 Quality Gate Logic
5.4 Trigger Strategy and Execution Modes
5.5 Reporting Artefacts
5.6 Traceability and Notification Layer
5.7 Metrics Dashboard Deployment

Introduction

The framework built in Chapter 4 runs in a Docker container on GitLab CI/CD, where it acts as the smoke-first gate before the regression suite. This chapter follows the pipeline from the inside out: the image the suite runs in, the stages and the gate that run it, and the reporting layers that surface each outcome.

5.1 Containerisation and Test Image

The framework's runtime bundles a Python interpreter, Robot Framework, the Browser Library, the Playwright Node.js runtime, and a specific Chromium build. A locally-installed runtime works for development, but CI demands more. Each pipeline run must be reproducible regardless of which runner picks up the job. Any toolchain drift would surface as intermittent failures that look like product defects.

We therefore package the entire runtime into a single Docker image, built by a dedicated pipeline stage and pushed to the GitLab project registry. We built it from scratch rather than reusing an upstream image. The bundled toolchain therefore matches the framework's exact requirements and stays under direct version control. The test job pulls this image and executes `robot` or `pabot` inside it. The shared runner's installed software has no effect on the result. The base layer is a Python distribution matching the project's pinned version. Playwright browsers are pre-installed during the build, and headless mode is the default. The same image is usable locally to reproduce any CI run.

The image is rebuilt and re-pushed only when its inputs change. Its build stage is gated behind a manual or scheduled trigger, so an incidental commit on the test sources does not trigger a registry push. Every other pipeline run reuses the cached image already published in the registry, and the test stage pulls the latest tag as-is.

5.2 Pipeline Implementation

The project runs three pipeline shapes, designed and modelled in Chapter 3 (Figures 3.5 and 3.6): the merge-request lint gate, the full web and scheduled pipeline, and the stability probe pipeline. All three are built from one GitLab CI/CD stage matrix [20].

The pipeline is structured around five sequential stages, plus an auto-injected sixth stage that GitLab Pages adds when the dashboard is published. Each event the pipeline reacts to activates a different subset, and only the stages relevant to that event run. Table 5.1 lists the stages, their jobs, and the events that activate them.

Stage	Job(s)	Role and activation
build	`quality-check`	Static linting of Robot Framework sources via Robocop on merge request events; results published as a GitLab Code Quality artefact for inline diff annotations.
build-image	`build-test-image`	Rebuild of the test container image and publication to the project registry; runs on manual and scheduled runs only.

test	`run-tests-parallel`
	`run-tests-sequential`
	`metrics-probe`
		Execution of the smoke suite in parallel or sequential mode (mutually exclusive). The `metrics-probe` job lives in the same stage but runs the stability probe; it is manual on web pipelines and automatic on probe schedules.

publish	`pages`	Appends one row to `history.json`, refreshes coverage, commits the update back to `main` with `[skip ci]`, and packages `dashboard/` into `public/` for deployment.
notify	`notify-teams` `trigger-e2e-pipeline`	Posts the Adaptive Card to the QA Microsoft Teams channel and fires the downstream regression pipeline once the smoke jobs report success.
deploy	*`pages:deploy`*	GitLab-injected stage that publishes the `public/` artefact from the `pages` job to the project's GitLab Pages site. Not declared in `.gitlab-ci.yml`.

Table 5.1: Pipeline stages, jobs, and activation rules

The same matrix yields the three shapes named above. The merge-request pipeline runs the quality gate only. The web or scheduled pipeline runs the remaining four stages plus the auto-injected `deploy`. The stability-probe pipeline reuses the `test` stage but bypasses the gate, and is covered in Section 5.7.2. The first two are described next.

5.2.1 Merge-Request Pipeline

The merge-request gate is a single `quality-check` job. It runs Robocop across `pages/`, `utils/`, and `tests/`, and publishes the violations as inline diff annotations in the GitLab merge-request interface. The lint result is signal-only, not enforcement: a violation does not auto-reject the merge request, but it surfaces in code review where the project's conventions require resolution before approval. Smoke tests deliberately do not run on merge requests, which keeps merge-request feedback fast and reserves the heavier stages for events where executing the suite is meaningful.

5.2.2 Web and Scheduled Pipeline

The web and scheduled pipeline activates the four declared stages above (`build-image`, `test`, `publish`, `notify`) plus the auto-injected `pages:deploy`, with the per-stage roles

given in Table 5.1. The gating relationship between the smoke run and the regression suite is the central operational property of this pipeline and is detailed in the next section.

5.3 Quality Gate Configuration

The whole pipeline lives in one `.gitlab-ci.yml` file. It opens with a `spec:inputs:` block declaring the four trigger inputs; these are re-exposed as variables so the `workflow` rules can read them and name each run. The five stages follow, then one job per role. Two YAML anchors keep the file concise: `.detect-env` resolves the target environment from `BASE_URL` for every test job, and `.commit-history` holds the commit-back logic shared by the `pages` and `metrics-probe` jobs. The gate itself is the `trigger-e2e-pipeline` job. Its `rules` fire the downstream regression only when the test stage succeeds on a web or scheduled run, skip it on smoke failure so regression never starts, and exclude it on probe runs. Listing 5.1 shows the stage declaration with the `quality-check` and `trigger-e2e-pipeline` jobs.

Listing 5.1: Pipeline excerpt from .gitlab-ci.yml

```
stages:
  - build
  - build-image
  - test
  - publish
  - notify

quality-check:
  stage: build
  script:
    - robocop check --gitlab
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      when: always
    - when: never
  artifacts:
    reports:
      codequality: robocop-code-quality.json
    expire_in: 7 days

trigger-e2e-pipeline:
  stage: notify
  trigger:
    project: lbc/e2e-tests
    branch: master
  variables:
    EXECUTE_AUTOMATICALLY: "NRTs"
    CUSTOM_URL: "$[[ inputs.BASE_URL ]]/"
    SKIP_SMOKE: "true"
  rules:
    - if: '$PIPELINE_KIND == "metrics-probe"'
      when: never
    - if: '$CI_PIPELINE_SOURCE == "schedule"'
      when: on_success
    - if: '$CI_PIPELINE_SOURCE == "web"'
      when: on_success
    - when: never
```

5.3.1 Fire-and-Forget Dispatch

The trigger uses GitLab's multi-project pipeline mechanism. When the smoke stage passes, `trigger-e2e` sends a downstream trigger to the regression project on its release branch and exits immediately, as described in Section 3.3.5. The downstream regression pipeline produces its own artefact set and its own Teams notification on a dedicated channel, independent of the smoke outcome.

5.3.2 Containment of False Positives

A smoke failure caused by infrastructure (a runner outage, a transient DNS delay, a brief environment restart) would otherwise block the regression suite for no reason. Two mechanisms contain this risk. First, the rerun mechanism described later in this chapter absorbs transient failures within the smoke run itself. Second, the gate is one-way: a smoke failure blocks regression, but it does not roll back any code or stop any merge that already happened. The cost of a false positive is a delayed regression run, not production impact. Chapter 6 shows the gate in action across a series of real pipeline runs.

5.4 Trigger Strategy and Execution Modes

A single pipeline definition serves both ad-hoc validation runs launched from the GitLab UI and recurring scheduled runs. The behavioural differences between the two emerge from inputs passed at trigger time rather than from separate pipeline files. Figure 5.1 shows the launch form exposed by GitLab when triggering the pipeline manually; Table 5.2 documents the four inputs and their defaults.

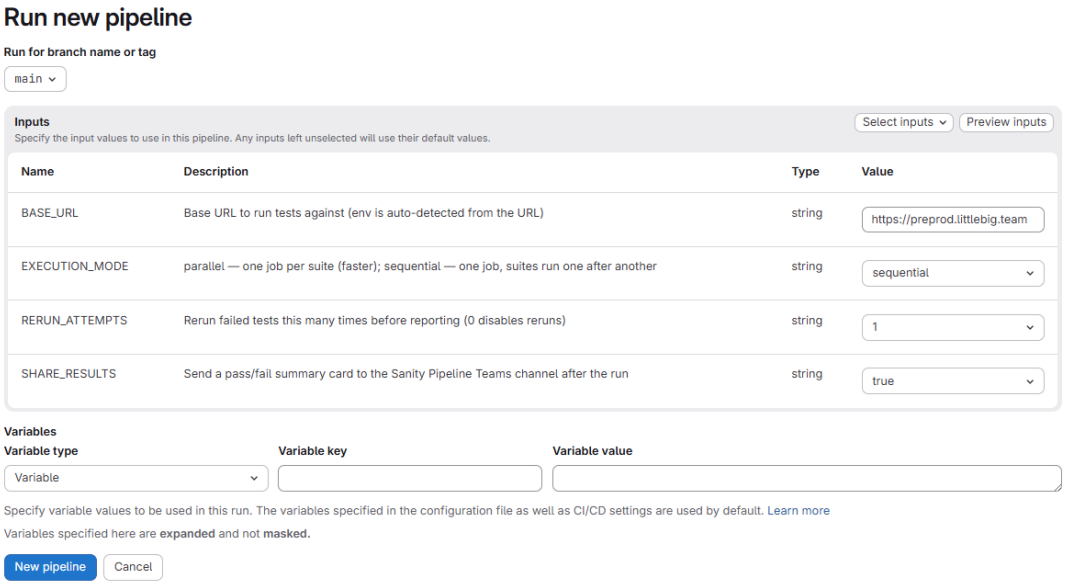


Figure 5.1: Pipeline launch form on GitLab

Input	Default	Effect
BASE_URL	the pre-production URL	Target environment URL. The pipeline routes credentials based on the detected environment; unrecognised URLs fail before tests start.
EXECUTION_MODE	sequential	parallel runs one job per domain in four concurrent runners; sequential runs one job that executes the four domains in order.
RERUN_ATTEMPTS	1	Number of automatic retries for failed tests within a single run. Accepts 0, 1, or 2.
SHARE_RESULTS	true	When true, the notify-teams job posts the Adaptive Card to the QA team channel after the run.

Table 5.2: Pipeline inputs exposed on the trigger form

5.4.1 Parallel and Sequential Execution Modes

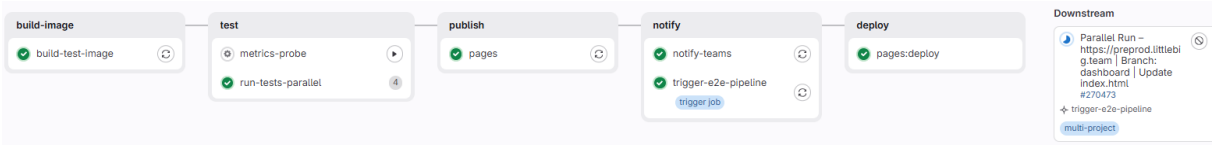
The sequential mode is the default trigger value, since most launches today are diagnostic in nature and benefit from a known, deterministic execution order. It runs a single job using `robot` (not `pabot`), iterating over the four domains in sequence and writing into `results/tmp-<suite>/` per domain. After all four complete, a single `rebot -name "Sanity Suite"` call merges them into a consolidated report under `results/`; only the merged artefacts are uploaded.

The parallel mode is opted into for scheduled nightlies and benchmark runs where wall-clock time matters more than execution ordering. Four test jobs run concurrently, one per domain, each launched with `pabot -processes N -testlevelsplit`. The process count is calibrated to the domain size: one for `tests/auth`, two for `tests/admin`, and four for both `tests/client` and `tests/supplier`. Each job writes into a per-suite results directory. A final `rebot -name <Display>` step then promotes the inner suite to root in each domain's report.

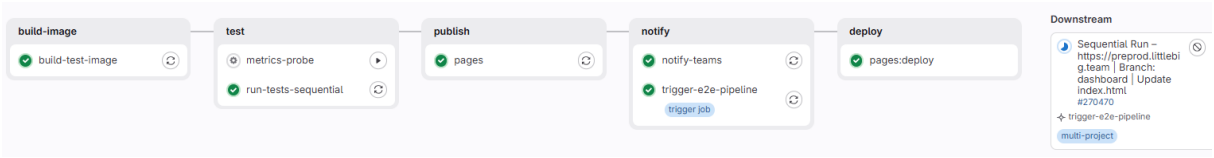
Both modes run the same suite lifecycle and produce identical final report shapes, including the JUnit-format `xunit.xml` (from `rebot -xunit`) that GitLab consumes for its native Tests tab. They differ only in runtime; the detailed comparison is in Chapter 6.

Figure 5.2 reproduces the in-pipeline view of a passing run in each mode, captured from the GitLab pipeline graph. Both pipelines show the same five-stage shape (`build-image` → `test` → `publish` → `notify` → auto-injected `deploy`) with the downstream `trigger-e2e-pipeline` job firing immediately after the smoke jobs pass. The single visible difference between the two is the `test` stage. In parallel mode it holds the

`run-tests-parallel` matrix job, with four concurrent runners collapsed into a single tile and a process count badge. In sequential mode it holds the single `run-tests-sequential` job.



(a) Parallel mode: the `run-tests-parallel` matrix job collapses four concurrent runners into a single tile.



(b) Sequential mode: a single `run-tests-sequential` job replaces the matrix.

Figure 5.2: Pipeline graphs in parallel and sequential modes

5.4.2 Flakiness Mitigation through Reruns

The `RERUN_ATTEMPTS` input absorbs transient infrastructure noise. After the initial test execution, any failed test is re-executed up to the configured number of times, and the merged report records the final status. Tests that pass on retry are reported as PASS in the final output. The audit lines in the Robot Framework log still preserve the original failure messages. This makes recovery rates visible to a human reader without polluting the pass/fail signal that drives the gate.

The default of one attempt was chosen after weighing the trade-off between flakiness tolerance and runtime cost. Disabling reruns (`RERUN_ATTEMPTS=0`) produces the cleanest signal but maximises sensitivity to transient blips. Two attempts (`RERUN_ATTEMPTS=2`) gives the strongest tolerance but extends the runtime when failures occur in bulk. One attempt absorbs the dominant class of transient failures without masking real defects. The rerun behaviour in practice is illustrated in Chapter 6 alongside the full gate effectiveness results.

5.4.3 Environment Routing

QA at LBC works with three categories of deployed environment:

- **ArgoCD.** Squad-private; each business squad runs its own instance to validate in-progress work without disturbing the others.
- **Pre-production (PPD).** The single shared environment where releases are ex-

ercised before they reach customers.

- **Production.** Reached only once pre-production passes its acceptance checks.

The smoke suite must route credentials to whichever of these the pipeline targets, without hard-coding any URL or password.

Routing is handled through GitLab CI/CD variables. Two variable families are declared, one for pre-production and one for ArgoCD, each holding the UI username, the UI password, and the legacy backend password used by the API fast path. The pipeline reads `BASE_URL` at startup, dispatches to the matching family, and exports the resolved values as `USERNAME`, `PASSWORD`, and `API_PASSWORD` for the test job. Unrecognised URLs fail before any test runs, preventing an accidental misconfiguration from authenticating with the wrong account.

The pipeline name is composed dynamically from these inputs: `<Mode> Run - <BASE_URL> | Branch: <branch> | <commit title>`. This lets the pipeline list surface intent and environment at a glance, without needing to open individual runs.

5.5 Reporting Artefacts

Each run emits a structured artefact set that the GitLab UI and downstream consumers read directly, listed in Table 5.3. Most are `robot` and `rebot` by-products; two are derived for GitLab's native integrations.

Artefact	Role
`output.xml`	Canonical machine-readable run record; the input for the Chapter 6 metrics scripts and for GitLab's Code Quality and JUnit consumers.
`report.html`, `log.html`	Human-readable summary plus a keyword-by-keyword trace, kept as-is from Robot Framework.
Screenshots and video	Failure captures from Browser Library, with Playwright video on failed tests, for replaying the browser state that failed.
`robocop-code-quality.json`	Code Quality report from the lint stage, rendered as inline annotations in the merge-request diff.
`xunit.xml`	JUnit format from `rebot -xunit`; populates GitLab's native Tests tab.

Table 5.3: Run artefacts and their consumers

5.6 Traceability and Notification Layer

A run that produces a comprehensive HTML report nobody opens is functionally equivalent to a silent failure. The traceability layer closes that loop by pushing structured outcomes from each pipeline run into the team's working channel.

5.6.1 Notification Stage

A dedicated `notify` stage runs after the test stage and before the regression trigger. It holds a single job that reads the test outcome and posts a result card to the QA team's Microsoft Teams channel. The job is non-blocking: a webhook outage, an unreachable Teams endpoint, or an unset webhook variable produces a soft failure that leaves the pipeline status unchanged. Notification is observability on top of the gate, never an override of it.

5.6.2 Adaptive Card Layout

The card follows Microsoft's Adaptive Card schema for Teams [21] and encodes the run outcome at three levels of detail, with the sections appearing in the order listed in Table 5.4.

Section	Content
Header	Pipeline identifier, environment label, branch, and run author.
Status banner	Coloured container with a status emoji (pass, fail, partial, neutral).
Overall summary	Passed and total counts with pass rate, failed count, total duration.
Suite results	One linked entry per executed suite job, with its own failure count.
Failed tests (collapsible)	Up to ten failure names with truncated messages, hidden by default.
Action button	Opens the pipeline page on GitLab in one click.

Table 5.4: Adaptive Card sections in display order

The environment label is computed from `BASE_URL`: pre-production surfaces as `PREPROD`, ArgoCD environments surface with their subdomain attached. The run author is resolved from the trigger source. Scheduled runs display `Scheduled` rather than the schedule owner's name, so an automated run does not falsely attribute itself to the developer who owns the cron entry. Failure messages are stripped of Robot Framework's embedded HTML and truncated to a fixed character budget. This keeps the card under Microsoft Teams' size limits while preserving enough context to triage.

Chapter 6 shows the Teams channel and the Adaptive Card in its three states: a passing parallel run, a passing sequential run, and a failed run with the collapsible block expanded.

5.6.3 Data Sources and Triggers

The notify job has two data sources for the same payload: the GitLab API and the JUnit-format artefacts produced by the test job. It tries the API first. When the job token lacks read access to the test report endpoint, a common case on shared GitLab instances, the API returns a structurally valid response with zero rows. The script then falls back to parsing the JUnit artefacts. The `SHARE_RESULTS` input (default `true`) is the user-facing opt-out for ad-hoc runs. Merge-request pipelines are explicitly excluded, since on that path only the lint job runs.

5.6.4 Downstream Hand-off

The downstream regression pipeline owns a separate Teams channel and posts its own start-of-run card there. The smoke pipeline does not duplicate this message in the QA channel, which keeps each channel focused on its own outcomes while the GitLab pipeline list view shows both runs side by side for anyone who needs the consolidated picture. Chapter 6 reproduces the downstream card alongside the full gate effectiveness results.

5.7 Metrics Dashboard Deployment

After every web or scheduled run on `main`, a single `pages` job updates the dashboard and publishes it. It does two things in one job: it appends the run's metrics row to `history.json`, then copies `dashboard/` into the `public/` directory GitLab Pages serves. The two steps stay in one job on purpose, because GitLab's Pages mechanism makes it awkward to hand the freshly-written `history.json` from one job to the next.

5.7.1 Job Steps

The job runs four steps in sequence:

1. **Install dependencies.** The publishing scripts need `PyYAML`; `git` is already in the test-runner image.

2. **Aggregate metrics.** `publish_metrics.py` parses the upstream Robot Framework artefacts, extracts the execution time and per-suite breakdown, refreshes coverage, and appends a row to `history.json` (schema in Section 4.7).

3. **Commit back.** The updated file is pushed to `main` with a `[skip ci]` flag to

avoid a pipeline loop. This needs the `METRICS_BOT_TOKEN` variable (a Project Access Token with `write_repository` scope) and is skipped silently when it is absent.

4. **Package for Pages.** `dashboard/` is copied into `public/`, which the auto-injected `pages:deploy` stage serves at the project's GitLab Pages URL.

The job carries `allow_failure: true`, so a metrics outage never affects the gate result.

5.7.2 Stability Probe Pipeline

The stability probe runs in a dedicated pipeline shape that bypasses the smoke gate. It is implemented by the `metrics-probe` job in the `test` stage. That job drives `infra/run_stability_probe.sh` to execute the smoke suite `R` times in declared order and `R` times in randomised order. It then invokes `traceability/analyze_probe.py` to append one row to `stability_probes` rather than `pipeline_runs`. The shared commit-back logic pushes the updated `history.json` to `main`. The `pages` job in the *same* pipeline then re-clones the repository, picks up the new probe row, and redeploys `public/`. The dashboard therefore reflects the probe inside the pipeline that produced it. The `pages` job's `needs: metrics-probe` declaration enforces this ordering. Figure 5.3 captures one such run as it appears on GitLab.



Figure 5.3: Stability-probe pipeline on GitLab

Probes are launched in one of two ways. On a manually-triggered web pipeline, the `metrics-probe` job is `when: manual` and fires only when the operator clicks its play button. Scheduled probes are driven by two GitLab Pipeline schedules. Their cron expressions avoid the Friday daytime busy window and the Sunday-evening downstream NRT slot, giving six probes per weekend (Figure 5.4). Both schedules set `PIPELINE_KIND=metrics-probe` and `EXECUTION_MODE=sequential`, which is what activates the probe-only routing through the pipeline rules.

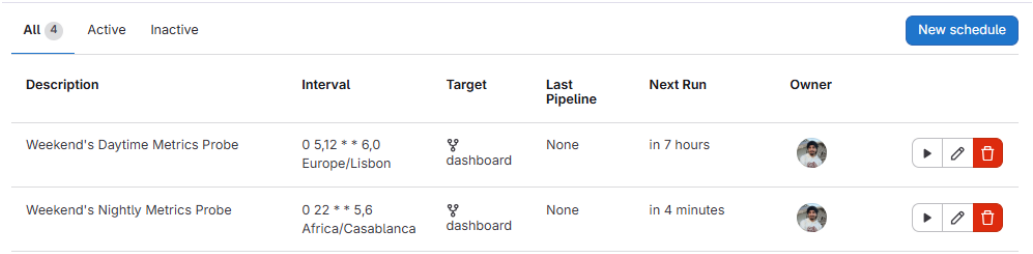


Figure 5.4: Weekend probe schedules on GitLab

Conclusion

The suite runs in production as the first stage of LBC's CI/CD pipeline. The container image fixes the runtime, the gate blocks the regression suite on smoke failure, and the traceability layer posts structured results to the QA team's Teams channel after every run. Chapter 6 measures whether it meets the targets set in Chapter 1.

Chapter 6

Results and Evaluation

Summary

6.1 Evaluation Methodology
6.2 Execution Time Benchmark
6.3 Flakiness Measurement
6.4 Gate Effectiveness and Coverage
6.5 Limitations and Threats to Validity

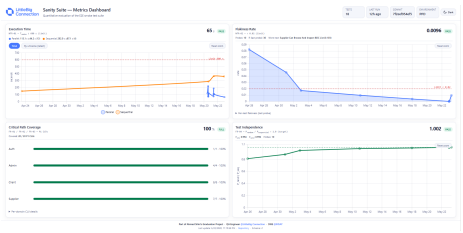
Introduction

This chapter evaluates the smoke suite against the targets set in Chapter 1. We define each metric before measuring it, so every dashboard number traces back to how it was computed and what counts as a pass.

6.1 Evaluation Methodology

The four evaluation criteria come from the non-functional requirements in Chapter 3 and the success metrics in Chapter 1: execution time (NFR-01, under ten minutes), flakiness (NFR-02, under 2%), test independence (FR-03, ratio 1.0), and critical path coverage (FR-01/02).

All four metrics are persisted in `dashboard/data/history.json` and rendered on the GitLab Pages dashboard. The rendered dashboard in both themes is reproduced in Figure 6.1, and Figure 6.2 captures the mobile breakpoint where the four cards collapse into a single scrolling column.

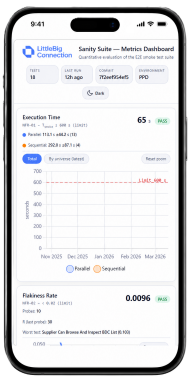


(a) Light theme



(b) Dark theme

Figure 6.1: Metrics dashboard in web view



(a) Mobile, light theme



(b) Mobile, dark theme

Figure 6.2: Metrics dashboard in mobile view

6.2 Execution Time Benchmark

The first target is the ten-minute execution budget. We compare the new smoke suite against the equivalent scope of the Selenium-based regression suite, run against the same pre-production environment with the same accounts.

6.2.1 Metric Definition

The execution-time requirement (NFR-01) is a one-sided inequality on the wall-clock duration of a smoke run:

$$T_{\text{smoke}} \leq T_{\text{target}}, \qquad T_{\text{target}} = 600\,\text{s}.$$

Pipeline runtimes fluctuate with runner load and backend response, so we report T_{smoke} across a window of R pipeline rows through its mean and sample standard deviation:

$$\mu = \frac{1}{R}\sum_{i=1}^{R} T_{\text{smoke},i}, \qquad \sigma = \sqrt{\frac{1}{R-1}\sum_{i=1}^{R}(T_{\text{smoke},i} - \mu)^2}.$$

Both quantities are split by execution mode on the dashboard, since parallel and sequential runs populate distinct distributions.

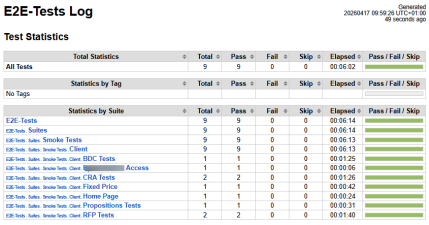
6.2.2 Per-Domain Comparison

Per-domain execution times were measured against pre-production during the implementation phase and aggregated in Table 6.1. The Selenium baseline reflects the matched scope (the same critical paths, run with the same input fixtures), not the full Selenium regression suite, which covers substantially more tests. Selenium has no standalone login test: it logs in as setup, so that cost is already inside each domain figure. The Auth row therefore shows the new suite's login test alone; the matched total compares the two stacks across the other 17 of the suite's 18 tests (Admin, Client, and Supplier).

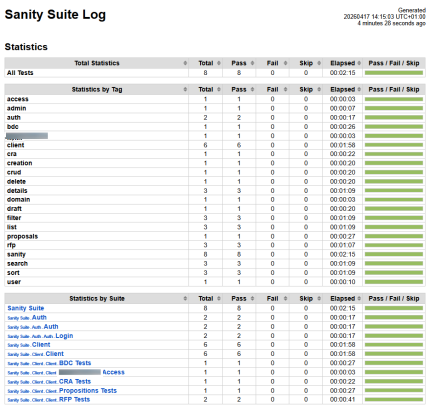
Universe	Tests	Selenium	RF with Browser Library	Ratio
Auth	1	–	0:19	–
Admin	4	1:35	0:41	2.3×
Client	6	6:13	1:58	3.2×
Supplier (incl. Freelancer)	7	6:23	1:47	3.6×
Matched total	**17**	**14:11**	**4:52**	**2.9×**

Table 6.1: Per-domain runtime, Selenium against RF with Browser Library. Auth, the suite's one login test, has no scope-matched Selenium counterpart, so it is shown for reference and excluded; the matched total covers the other 17 of the 18 tests.

Ratios range from 2.3× to 3.6×. The largest gains appear on AgGrid-driven flows (RFP, BDC, CRA), where native element-state waits replace polling loops. Flows whose page-load wait already operated at the same granularity in both stacks gain the least. Figures 6.3, 6.4, and 6.5 reproduce the Robot Framework reports for the three largest domains.

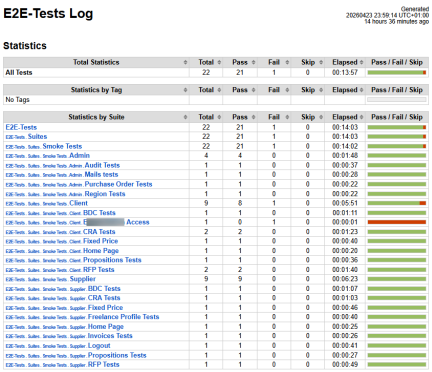


(a) Selenium baseline

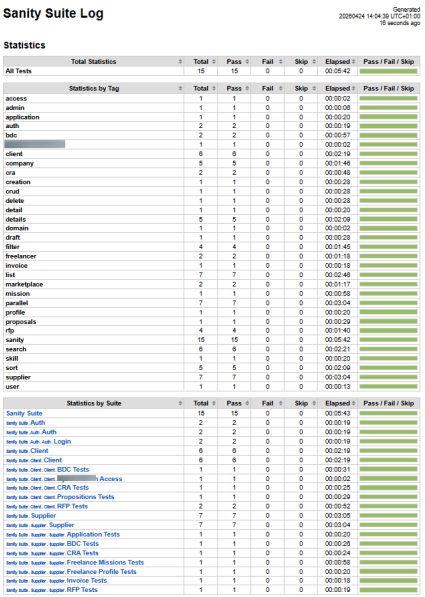


(b) RF with Browser Library smoke

Figure 6.3: Client domain runtime, Selenium against the smoke suite

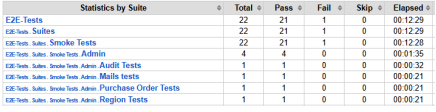


(a) Selenium baseline

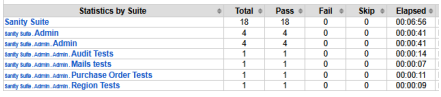


(b) RF with Browser Library smoke

Figure 6.4: Supplier domain runtime, Selenium against the smoke suite



(a) Selenium baseline



(b) RF with Browser Library smoke

Figure 6.5: Admin domain runtime, Selenium against the smoke suite

A clean Selenium run takes roughly 14 minutes across the matched scope, against a steady-state sequential mean of about 4 min 52 s (∼292 s) for the new suite, a 2.9× reduction. The new suite sits well inside the ten-minute budget, and parallel mode cuts it further. Selenium is itself flaky, swinging from a clean run near 13 minutes to over 20. The speed-up ratio is therefore the stable figure, not the absolute seconds. The new suite's per-pipeline distribution is in the evolution view below.

Part of that headroom is the authentication fast path of Section 4.3: the first login per Robot process runs through the UI at ∼7.6 s, and every later login replays the cached session at ∼1 s. With one UI login per worker, login is no longer a measurable share of the total.

6.2.3 Parallel versus Sequential Mode

The pipeline offers two execution modes, and the choice between them trades concurrency cost against wall-clock time. Table 6.2 reports the total runtime each mode produces against pre-production.

Mode	Total runtime (pre-prod)	Within 10-min budget?
Parallel (4 domain jobs)	∼ 1:53	Yes
Sequential (single job)	∼ 4:52	Yes

Table 6.2: Smoke suite total runtime by execution mode

The parallel mode trims roughly three minutes off the sequential total by overlapping the four domain jobs. Neither mode threatens the ten-minute target. Sequential is the default trigger value and dominates day-to-day diagnostic launches; parallel is opted into to minimise wall-clock time on scheduled and benchmark runs.

Outside CI, the suite runs locally with the Playwright Inspector attached (Figure 6.6). The Inspector overlays the current locator on the live page. It lets the author step through a test and pause or resume it. A selector can be edited against the live DOM until it matches. The run can be driven in slow motion to watch each action. Every run leaves a video to replay. The trace viewer adds time-travel debugging, with a DOM snapshot at each step. Together these keep the locator policy (CSS or `id` only, every locator unique) enforceable during authoring and make a failure quick to diagnose.

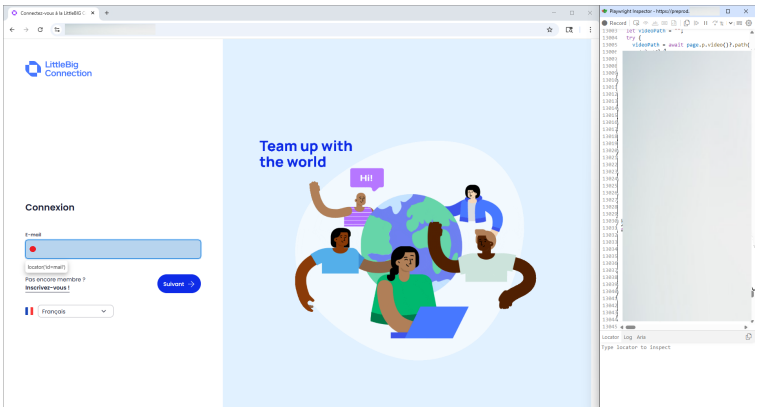
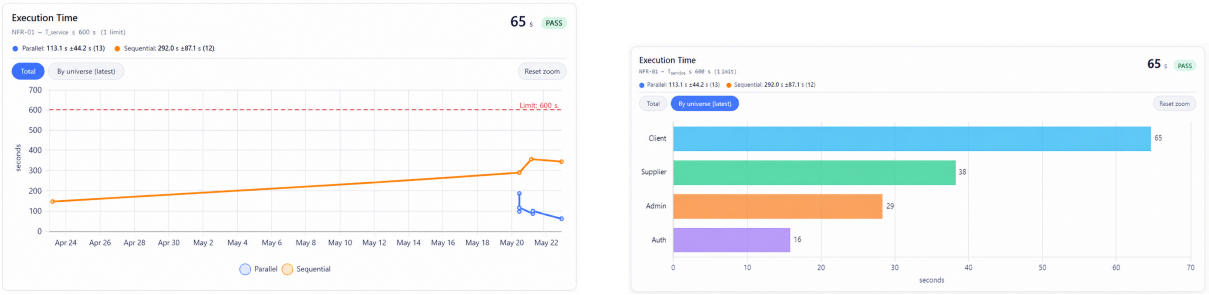


Figure 6.6: Playwright Inspector during local debugging

6.2.4 Evolution Across Pipeline Runs

A single benchmark is a snapshot; the dashboard tracks the metric across every pipeline so drift becomes visible. Figure 6.7 reproduces the execution-time card. The two modes produce sharply distinct distributions: $\mu_{\text{par}} = 113.1\,\text{s}$ ($\sigma = 44.2\,\text{s}$, $n = 13$); $\mu_{\text{seq}} = 292.0\,\text{s}$ ($\sigma = 87.1\,\text{s}$, $n = 12$). Even at $\mu + \sigma$, sequential stays under 379 s and parallel under 158 s, both comfortably below the 600-second limit.



(a) Total view (b) By domain (latest run)

Figure 6.7: Execution-time card

6.3 Flakiness and Test Independence

A two-percent flakiness target is meaningless without a defined measurement protocol. Ours is the stability probe orchestrated by `infra/run_stability_probe.sh`.

6.3.1 Metric Definitions

The flakiness rate (NFR-02) aggregates per-test instability into a single suite-level number. With $N = 18$ tests (the sanity-tagged P1 set), R consecutive executions against the same build, and f_t the number of pass-to-fail or fail-to-pass transitions observed for test t across those R runs:

$$\text{Flakiness} \;=\; \frac{1}{N}\sum_{t=1}^{N}\frac{f_t}{R-1}.$$

The inner ratio scores each test in $[0, 1]$; averaging over N gives the suite-level rate, with the target requiring it to stay below 0.02. The test-independence ratio (FR-03) compares the suite's pass rate between two orderings:

$$\text{Independence} \;=\; \frac{P_{\text{random}}}{P_{\text{sequential}}}.$$

$P_{\text{sequential}}$ is the mean pass rate over R runs in declared order; P_{random} is the mean over R runs with `--randomize tests`. A ratio of 1.0 indicates full order independence; below 1.0 exposes hidden coupling.

6.3.2 Probe Design

A probe pipeline runs the smoke suite `R` times sequentially then `R` times randomised against the same build, with default $R = 30$ (29 flip points per test). The `2R` run artefacts are parsed by `traceability/analyze_probe.py`, which appends one row to `stability_probes` in `history.json`.

6.3.3 Flakiness and Independence Results

Thirteen probes were captured between the framework's first stability run and the report's data freeze, each with $R = 30$. The first was the calibration baseline. After the test data strategy was tightened and the few coupled fixtures isolated, both metrics reached target and held there across eleven consecutive probes. Table 6.3 summarises three reference points; the full trend is on the dashboard cards (Figures 6.8 and 6.9).

Probe	R	Suite flakiness	Independence ratio	Target met?
First probe (Apr 26)	30	0.083	0.810	No (both above / below)
Mid-window (May 2)	30	0.018	0.920	Flakiness met; coupling residual
Latest probe (May 24)	30	0.0077	1.004	**Both met**

Table 6.3: Flakiness and independence at three reference probes

The latest probe sets the steady-state baseline: two Supplier tests (BDC and CRA list browsing) flipped twice each over thirty runs ($f_t = 2$, per-test rate $2/29 \approx 0.069$), giving a suite-level flakiness of $2 \times 0.069/18 = 0.0077$, well below the 0.02 target. Independence reached 1.004 ($P_{\text{seq}} = 0.996$, $P_{\text{random}} = 1.000$); any sustained drop below 1.0 on later probes flags a coupling regression.



(a) Suite-level flakiness rate per probe



(b) Per-test breakdown for the latest probe

Figure 6.8: Flakiness card



Figure 6.9: Test-independence card

6.4 Gate Effectiveness and Coverage

This section measures the gate the suite implements: how completely it covers the critical journeys catalogued in Chapter 3, and how often it caught a real breakage.

6.4.1 Critical Path Coverage

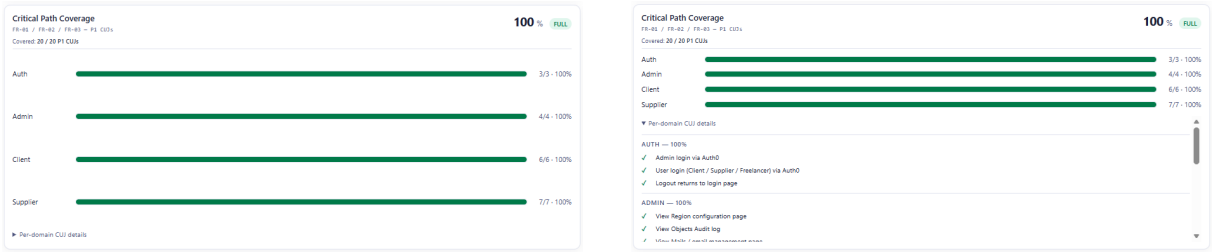
For each domain $d \in \{\text{Auth}, \text{Admin}, \text{Client}, \text{Supplier}\}$, let C_d be the priority-one journeys catalogued for that domain and $A_d \subseteq C_d$ the subset automated. Then:

$$\text{Coverage}_d = \frac{|A_d|}{|C_d|}, \qquad \text{Coverage}_{\text{global}} = \frac{\sum_d |A_d|}{\sum_d |C_d|}.$$

Coverage is computed from `traceability/cuj_catalogue.yaml` (the SSOT for priority-one journeys per domain) by `compute_coverage.py`. Table 6.4 gives the per-domain and per-tier breakdown, and the same figures are rendered on the dashboard's coverage card (Figure 6.10).

Domain	Automated	Catalogued	Ratio
Auth	3	3	100%
Admin	4	4	100%
Client	6	6	100%
Supplier	7	7	100%
P1 total	**20**	**20**	**100%**
P2 (all domains)	2	4	50%
P3 (all domains)	0	2	0%
Full catalogue	**22**	**26**	**84.6%**

Table 6.4: Critical path coverage per domain and across priority tiers



(a) Per-domain bar chart with global badge (b) Per-CUJ collapsible detail list

Figure 6.10: Critical path coverage card

The Key Performance Indicator (KPI) set in the specification was 100% coverage of the P1 set, and that target is met; the two opportunistically-automated P2s and the deferred P2/P3 entries are documented as the first perspective in the General Conclusion.

6.4.2 Gate Effectiveness

Gate effectiveness counts how often the smoke run blocks the downstream regression on a real critical-path breakage versus a false positive. A blocked regression saves the full multi-hour compute and triage cost of the downstream run; a false positive costs one delayed regression run. Figure 6.11 shows a failed smoke run holding the regression trigger closed.

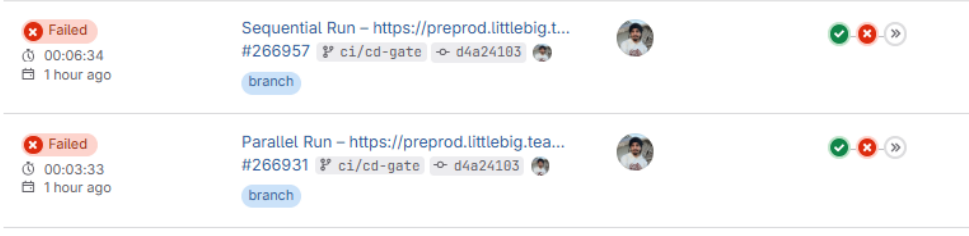
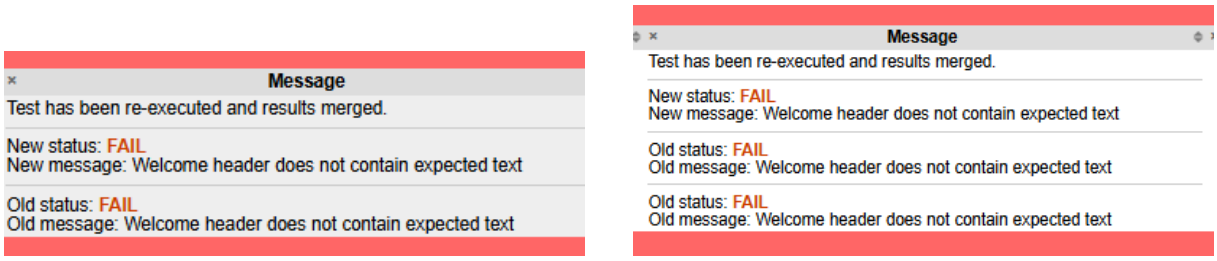


Figure 6.11: Failed smoke run blocking the regression trigger

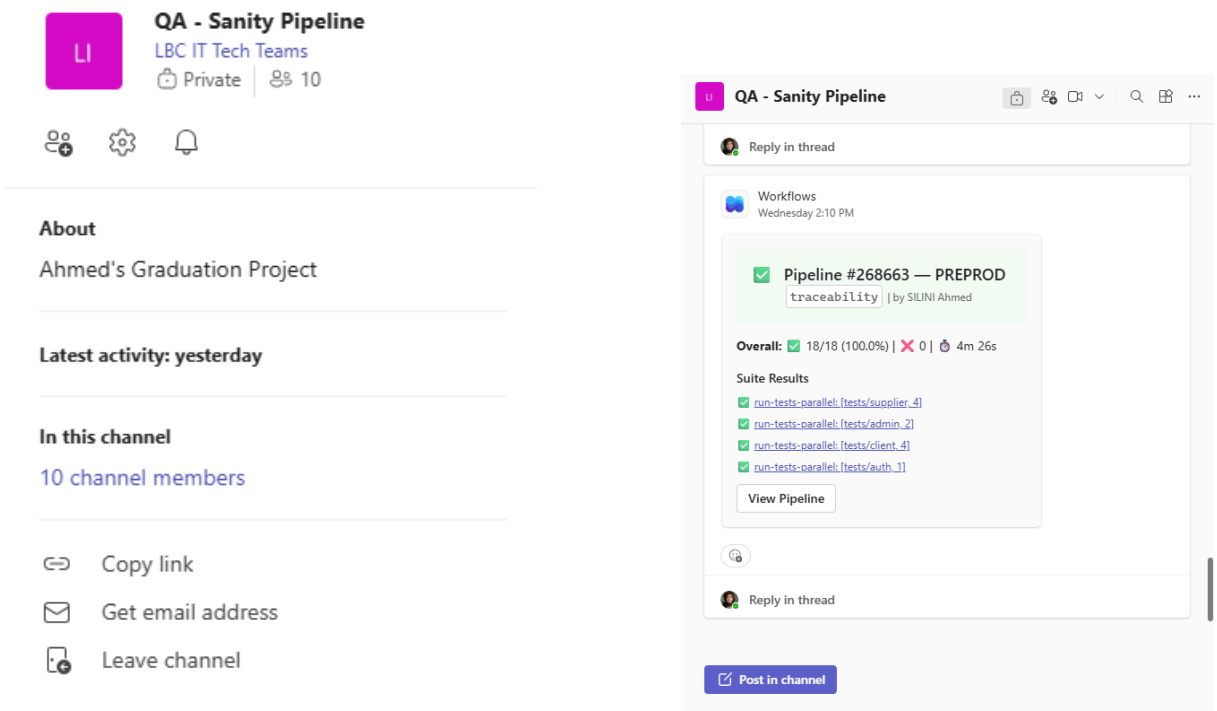
Figure 6.12 shows the rerun mechanism absorbing a transient failure: the first attempt marks one test as failed, the automatic rerun recovers it, and the merged report shows a clean pass.



(a) One rerun attempt: recovery trail visible (b) Two rerun attempts: worst-case budget

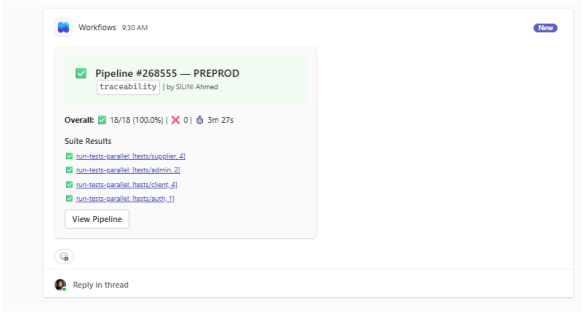
Figure 6.12: Rerun mechanism absorbing a transient failure

Figures 6.13 and 6.14 reproduce the QA team's Microsoft Teams channel feed and the Adaptive Card in its three states (parallel success, sequential success, failure with the failed-tests block expanded).

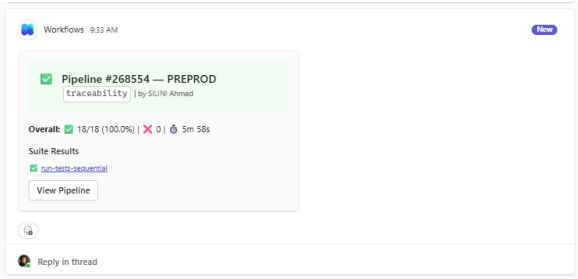


(a) Channel feed: card history (b) Channel interface

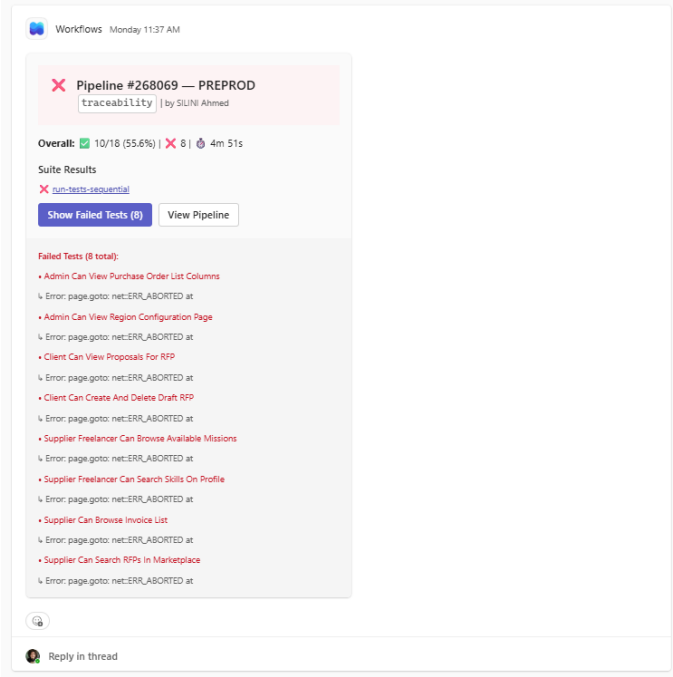
Figure 6.13: QA team Microsoft Teams channel



(a) Passing parallel run



(b) Passing sequential run



(c) Failed run, failed-tests block expanded

Figure 6.14: Adaptive Card in three states

Figure 6.15 shows the downstream regression pipeline announcing itself in its own Teams channel after a smoke success; neither channel duplicates the other.

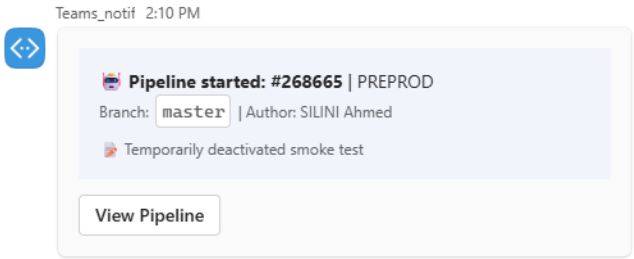


Figure 6.15: Downstream regression pipeline card in its own Teams channel

6.4.3 Remaining Requirements Verification

The four metrics above address FR-01 through FR-03 and NFR-01 through NFR-02 directly. Table 6.5 summarises the verification evidence for the remaining nine requirements.

Requiremer	Verification evidence
FR-04	UI and API fast paths implemented for Client, Supplier, Admin, and Freelancer roles; every smoke test uses one path without modification.
FR-05	`utils/env.resource` routes credentials from `BASE_URL`; switching environments requires only a different input value at trigger time.
FR-06	Smoke suite is the first gate stage in LBC's GitLab pipeline; `trigger-e2e` is blocked on smoke failure.
FR-07	Every run produces an RF HTML report, video recordings, HAR traces, and JUnit XML.
FR-08	The GitLab Pages dashboard tracks execution time, flakiness, coverage, and independence across runs, refreshed by `publish_metrics.py` on each pipeline.
FR-09	Every run posts a Teams Adaptive Card with its outcome, so the team sees pipeline health in real time.
NFR-03	Four-layer architecture and coding conventions enforced by Robocop on every merge request; no selector has crossed the pages/utils boundary since it was codified.
NFR-04	A new test requires only a `.robot` file and one `Resource` line in the aggregator; no change to utilities, lifecycle, or pipeline is required. Demonstrated during D6 when CUJ-A05 was reformulated from the data-export trigger to the purchase-order list test: only `tests/admin/Purchase_Order.robot` changed.
NFR-05	Same mechanism as FR-05: `BASE_URL` and `utils/env.resource` decouple the suite from any specific deployment target.

Table 6.5: Verification evidence for the remaining requirements

6.5 Limitations and Threats to Validity

The results above need to be read against the boundaries of what the suite exercises and how the measurements were captured.

- **Scope of the gate.** Twenty P1 journeys; the deferred P2/P3 entries and non-functional properties (load, accessibility, visual regression) stay outside the gate. A slow-but-correct page still passes.

- **Operational fragility.** Parallel mode saturates the shared runner pool (four jobs × four pabot workers), so it is reserved for scheduled runs. The API fast path tracks Auth0's silent-auth probe and SDK localStorage layout: an SDK

change silently reverts the suite to the UI path ($\sim 8\times$ login cost). Read-only tests share pre-production accounts that another team can rename, surfacing as a smoke failure even when the platform is healthy.

- **No framework self-testing.** Page-object resources and utility keywords have no unit tests. A silent bug in a shared utility (a grid-header lookup returning the wrong column, a vacuous assertion) could invalidate dependent tests without surfacing. Robocop catches structural drift, not semantic bugs.

- **Measurement windows.** Probes use $R = 30$ sequential and $R = 30$ randomised executions: enough for a suite-level baseline, but a single highly-flaky test can hide inside the mean (the per-test breakdown exists for that reason). The Selenium benchmark compares the same journeys under two stacks, against pre-production under typical load. Absolute timings shift with environment contention, but the relative speed-up and the gate's pass/fail behaviour generalise.

Integration of the smoke suite into the regression project, once the regression migration onto the new stack is complete, is documented as a perspective in the General Conclusion.

Conclusion

The smoke suite meets the execution-time target by a comfortable margin: approximately two minutes in parallel mode and just under five in sequential, against a ten-minute budget. The per-domain comparison against the matched Selenium scope shows a $2.3\times$ to $3.6\times$ reduction. Flakiness, independence, and coverage are tracked continuously on the dashboard, and the gate's blocking behaviour was verified on real pipeline runs. The QA team now has a fast-feedback gate that runs in front of the regression suite and reports its own outcome on its own timeline.

General Conclusion and Perspectives

This internship delivered a working smoke test suite that runs as the first stage of LittleBig Connection's CI/CD pipeline, in front of the existing Robot Framework regression suite. Eighteen priority-one tests automate twenty critical user journeys across Auth, Admin, Client, and Supplier. The full suite completes well inside the ten-minute target, a 2.9× reduction over the matched Selenium scope.

The contribution is more than the test code. It also includes a layered framework under the Page Object Model and Facade boundaries, a Docker test image, two pipeline shapes with a configurable rerun budget, a Microsoft Teams Adaptive Card, and a static GitLab Pages dashboard. The dashboard tracks execution time, flakiness, coverage, and independence over time.

The two most consequential design challenges were the Auth0 fast-path and the boundary between business logic in `pages/` and UI mechanics in `utils/`. The fast-path's one-second result is what makes per-test login viable. The `pages/` and `utils/` boundary is codified by the conventions in Chapter 4 and enforced by Robocop.

Six concrete improvements stand out as the most valuable next steps:

1. **Closing the catalogue's open P2/P3 entries.** Appendix A lists the deferred journeys: the P2 candidates (CUJ-C10 / CUJ-C11 *RFP search*, CUJ-A07 *Manage user accounts*, CUJ-A08 *Configure system notifications*) are the most natural additions to the smoke gate once their flows stabilise; the two P3 entries (CUJ-A05 *Data Export*, asynchronous backend; CUJ-AU04 *Create Account*, provisioning path covered by the regression suite) draw the intentional boundary.

2. **Xray traceability for smoke results.** The regression suite already feeds Jira via Xray; the smoke suite publishes only a Teams card and a GitLab Tests tab. The integration is straightforward: Xray accepts JUnit XML through its REST API, and the smoke pipeline already produces `xunit.xml`.

3. **Cross-universe smoke scenarios.** Two browser contexts in a single Playwright process, driving a Client action and a Supplier reaction inside one test, is exactly the capability for which the Browser Library stack was chosen over Cypress and SeleniumLibrary. The smoke gate's P1 set is composed entirely of single-universe

journeys, so the capability is engineered in but not yet exercised by a test. The cross-universe scenarios listed in P2 and beyond are its natural first use. They cannot be expressed under any other stack we evaluated.

4. **Alerting on metric drift.** The stability probe already runs on a weekly schedule (Chapter 5). The missing piece is an automatic alert when a tracked rate breaches the KPIs we set: flakiness above two percent, or independence away from 1.0. That turns the dashboard from a passive record into an active signal.

5. **Migration of the regression suite onto the Browser Library stack.** Sequenced as the structural refactor already in flight, followed by a squad-by-squad engine swap under existing QA-engineer ownership, and optionally consolidating the smoke suite into the same repository.

6. **Pushing parallelism further within each domain**, rather than only across domains, would trim another minute or two off the worst-case runtime as the suite grows.

Taken together, these steps keep the smoke suite useful as the platform scales, fold it into the same traceability fabric the regression suite already uses, and outline a credible path toward a broader migration of LBC's automated test estate onto the same stack.

Bibliography

[1] International Software Testing Qualifications Board. "Certified tester foundation level syllabus, version 4.0.1," Accessed: May 23, 2026. [Online]. Available: https://www.istqb.org/certifications/certified-tester-foundation-level

[2] L. Crispin and J. Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley Professional, 2009, ISBN: 978-0321534460.

[3] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional, 2009, ISBN: 978-0321579362.

[4] M. Fowler. "Test pyramid," Accessed: May 23, 2026. [Online]. Available: https://martinfowler.com/bliki/TestPyramid.html

[5] G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley Professional, 2007, ISBN: 978-0131495050.

[6] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010, ISBN: 978-0321601919.

[7] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016, ISBN: 978-1942788003.

[8] M. Fowler. "Continuous integration," Accessed: May 23, 2026. [Online]. Available: https://martinfowler.com/articles/continuousIntegration.html

[9] Selenium Project. "Selenium webdriver documentation," Accessed: May 23, 2026. [Online]. Available: https://www.selenium.dev/documentation/webdriver/

[10] Cypress.io. "Cypress documentation," Accessed: Jun. 8, 2026. [Online]. Available: https://docs.cypress.io/

[11] Robot Framework Foundation. "Robot framework user guide," Accessed: May 23, 2026. [Online]. Available: https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html

[12] Microsoft. "Playwright documentation," Accessed: May 23, 2026. [Online]. Available: https://playwright.dev/

[13] Robot Framework Foundation. "Robot framework browser library," Accessed: May 23, 2026. [Online]. Available: https://marketsquare.github.io/robotframework-browser/

[14] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016, ISBN: 978-0133943030.

[15] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002, ISBN: 978-0321127426.

[16] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021, ISBN: 978-0136886099.

[17] M. Seemann, *Code That Fits in Your Head*. Addison-Wesley, 2021, ISBN: 978-0137464401.

[18] M. Fowler. "Pageobject," Accessed: May 23, 2026. [Online]. Available: https://martinfowler.com/bliki/PageObject.html

[19] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994, ISBN: 978-0201633610.

[20] GitLab Inc. "Gitlab ci/cd documentation," Accessed: May 23, 2026. [Online]. Available: https://docs.gitlab.com/ee/ci/

[21] Microsoft. "Adaptive cards for microsoft teams: Documentation," Accessed: May 23, 2026. [Online]. Available: https://adaptivecards.io/

Appendices

Appendix A: Critical User Journey Catalogue (extract from D1)

Source: deliverable **D1 — Critical Path Analysis** (v1.1, March 2026, reviewed and validated with the QA team lead). Priorities and step summaries below have been re-aligned with the implementation. New journeys identified during build are listed alongside the original D1 entries: CUJ-C09 Partner Domain Access, CUJ-A06 Purchase Order list, and CUJ-A07 / CUJ-A08 candidate Admin flows. **P1** marks journeys automated in the smoke gate; **P2** marks candidates carried into the Perspectives section; **P3** marks journeys deferred to the regression suite.

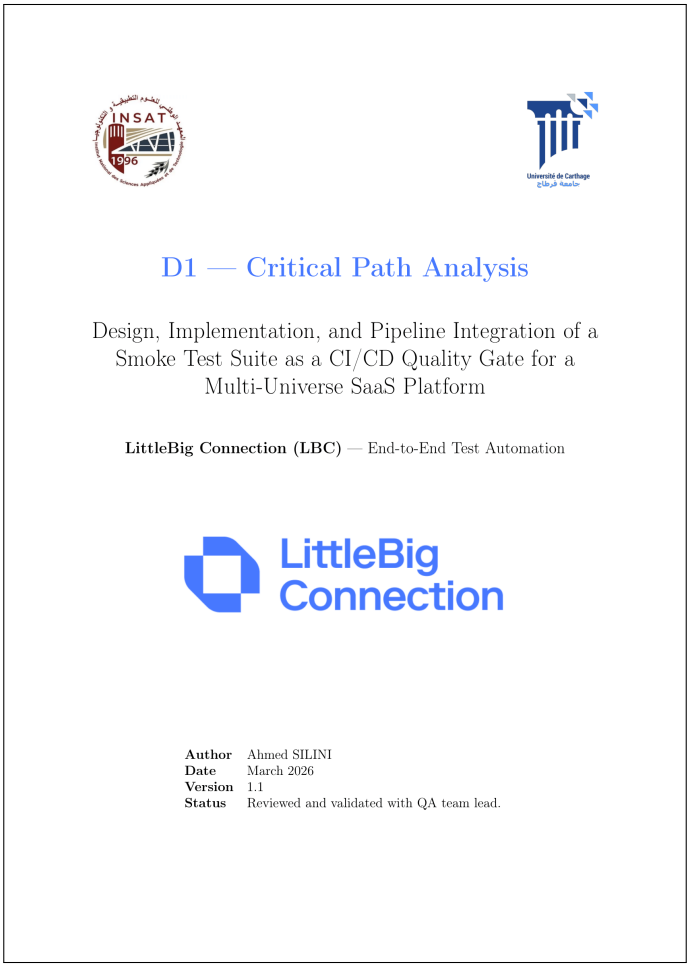


Figure A.1: Title page of D1.

Auth Domain

ID	Journey	Steps Summary	Pri.
CUJ-AU01	Admin Login	Navigate to admin login → Auth0 form → assert dashboard loads.	P1
CUJ-AU02	User Login (Client / Supplier / Freelancer)	Navigate to login → Auth0 form → assert role-specific landing page loads.	P1
CUJ-AU03	Logout	Trigger logout → assert redirect to login page.	P1
CUJ-AU04	Create Account	Open the account-creation form → submit identity & credentials → assert account is provisioned and login succeeds.	P3

Table A.1: Auth domain critical user journeys, extract from D1 v1.1

Client Universe

ID	Journey	Steps Summary	Pri.
CUJ-C02	RFP List and Detail (Referenced & Non-Referenced)	Access RFP list → apply sort and filter → open a referenced or non-referenced RFP from the Template or Draft tab → assert results render and the detail page loads with correct data. List navigation and detail rendering are catalogued as a single CUJ since they are validated end-to-end by one sanity test.	P1
CUJ-C05	Create and Delete Draft RFP	Navigate to RFP creation → fill required fields (entity, title, location, duration, budget) → save as draft → assert draft in list → delete → assert removal.	P1
CUJ-C06	BDC List and Detail	Access BDC list → sort and filter → open one BDC → assert detail loads.	P1
CUJ-C07	CRA List and Detail	Access CRA list → sort and filter → open one CRA → assert detail loads.	P1
CUJ-C08	Propositions List	Open an RFP → navigate to Propositions tab → assert list loads.	P1
CUJ-C09	Partner-Domain Access	Hit cross-tenant URL → retry up to three times → assert routing succeeds.	P1
CUJ-C10	RFP Search (Template page)	Open Template tab → enter a search query by reference → assert the matching RFP appears in the filtered results.	P2
CUJ-C11	RFP Search (Draft page)	Open Draft tab → enter a search query by reference → assert the matching draft RFP appears in the filtered results.	P2

Table A.2: Client universe critical user journeys, extract from D1 v1.1

Supplier Universe (including Freelancer flows)

ID	Journey	Steps Summary	Pri.
CUJ-S02	RFP Marketplace Search	Access RFP marketplace → enter search query → assert results display.	P1
CUJ-S03	Application List and Detail	Navigate to Applications → assert list loads → open one application → assert detail loads.	P1
CUJ-S04	CRA List and Detail	Access CRA list → sort and filter → open one CRA → assert detail loads.	P1
CUJ-S05	BDC List and Detail	Access BDC list → sort and filter → open one BDC → assert detail loads.	P1
CUJ-S06	Invoice List	Navigate to Invoices → assert list loads with correct table headers.	P1
CUJ-F02	Skill Search on Profile	Access Freelancer profile → SkillsAlly search → enter a skill query → assert autocomplete results appear.	P1
CUJ-F03	Browse Available Missions	Navigate to missions section → assert list loads.	P1

Table A.3: Supplier universe critical user journeys (including Freelancer flows), extract from D1 v1.1

Admin Universe

ID	Journey	Steps Summary	Pri.
CUJ-A02	Region Configuration	Navigate to region config → assert page loads with expected regions.	P1
CUJ-A03	Audit Log	Navigate to Audit section → assert log loads and contains entries.	P1
CUJ-A04	Mails Interface	Navigate to Mails section → assert interface loads.	P1
CUJ-A06	Purchase Order List	Navigate to admin Purchase Order list → assert PrimeVue table loads.	P1
CUJ-A07	Manage User Accounts	Open user management → create, edit, deactivate an account → assert state.	P2
CUJ-A08	Configure System Notifications	Open notifications config → enable a channel → assert delivery.	P2
CUJ-A05	Data Export Trigger	Trigger export → assert job is initiated.	P3

Table A.4: Admin universe critical user journeys, extract from D1 v1.1

Appendix B: Per-Domain Design Class Diagrams

The four per-domain diagrams take the analysis-level class model from Chapter 3 one level deeper. Each diagram shows the page-object resources that realise its domain. Auth is rendered from the PoC, and every later domain follows the same Page Object Model and Facade structure.

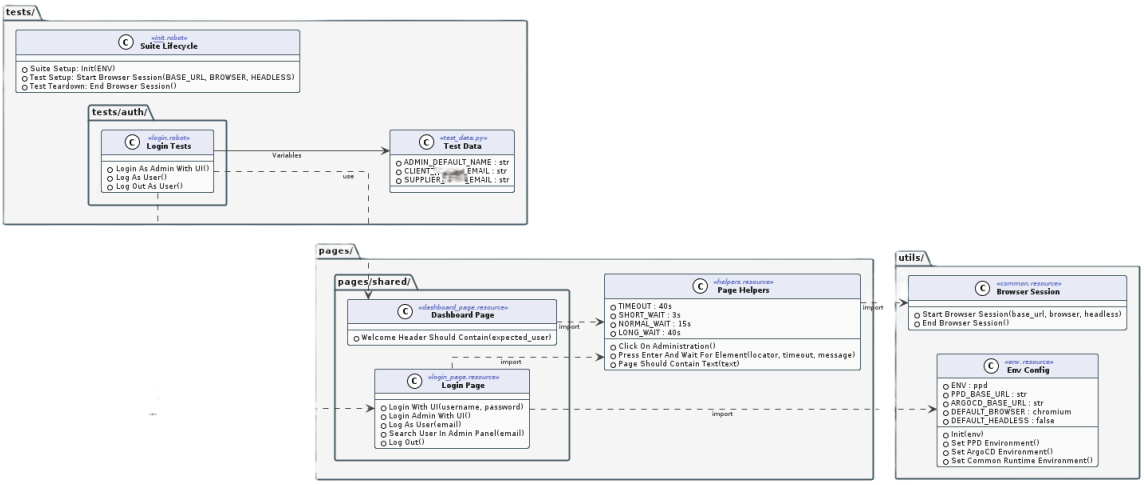


Figure B.1: Design class diagram: Auth domain (PoC)

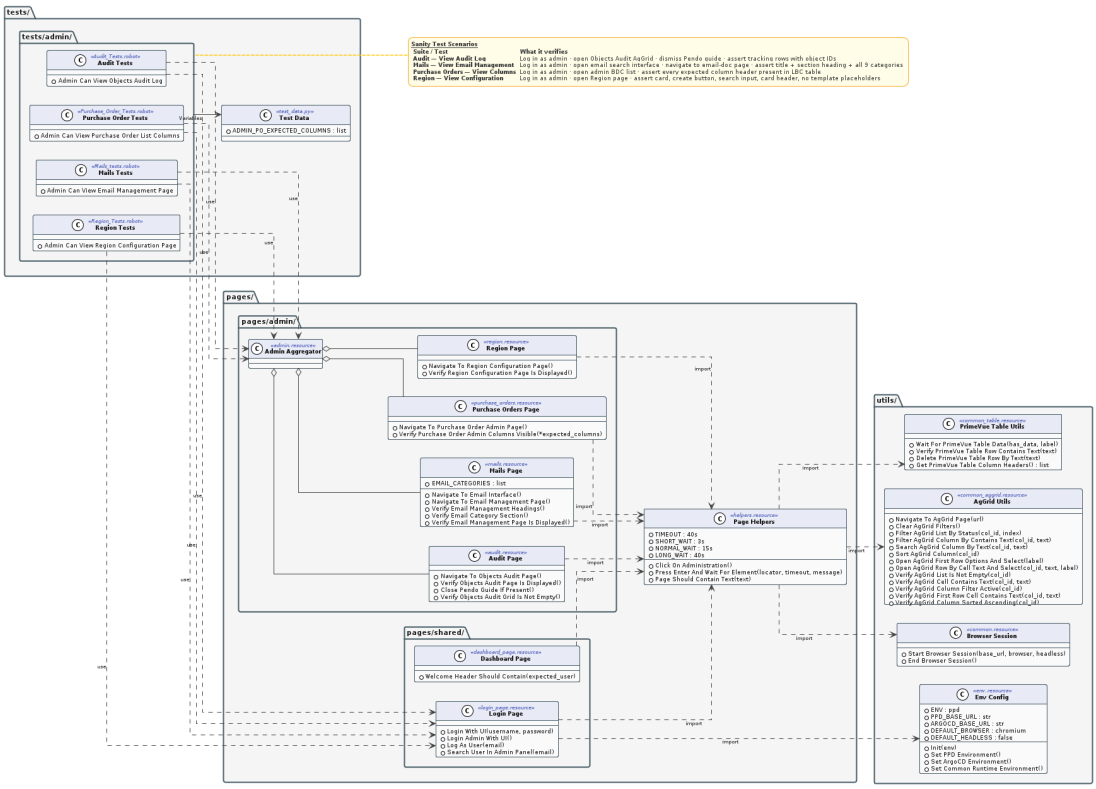


Figure B.2: Design class diagram: Admin domain

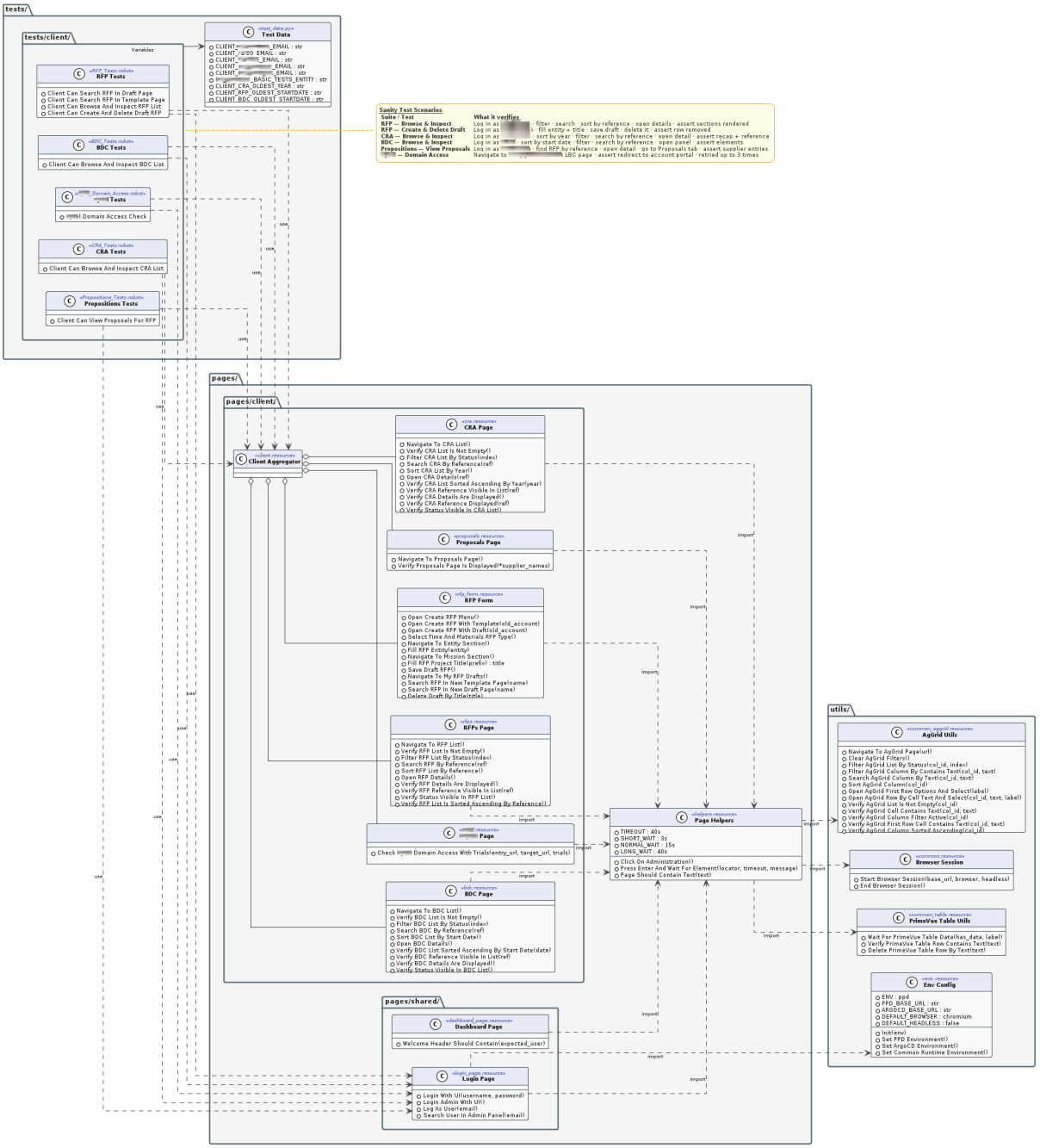


Figure B.3: Design class diagram: Client domain

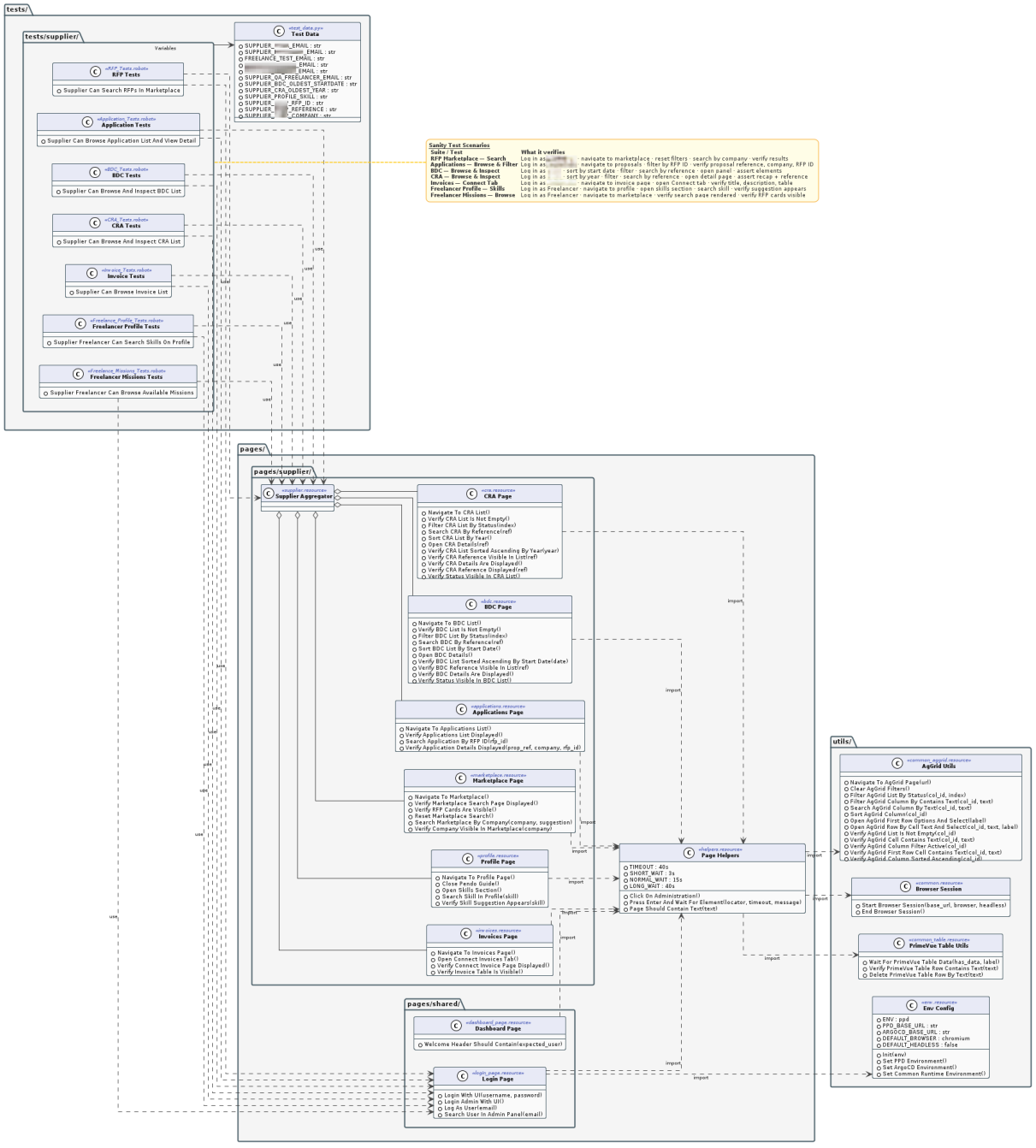


Figure B.4: Design class diagram: Supplier domain

Appendix C: Repository Layout

The full project tree at the end of the implementation phase is reproduced below for reference.

Listing C.1: Project tree (detailed)

```
sanity-suite/
|-- infra/
|   |-- dockerfile
|   |-- run_tests.sh
|   |-- run_parallel_suite.sh
|   `-- run_sequential_suites.sh
|-- traceability/
|   |-- teams_notification.py
|   |-- publish_metrics.py
|   |-- analyze_probe.py
|   |-- compute_coverage.py
|   `-- cuj_catalogue.yaml
|-- dashboard/
|   |-- index.html
|   |-- app.js
|   |-- styles.css
|   |-- assets/
|   `-- data/
|       `-- history.json
|-- utils/
|   |-- env.resource
|   |-- common.resource
|   |-- libraries.resource
|   |-- common_aggrid.resource
|   `-- common_table.resource
|-- pages/
|   |-- helpers.resource
|   |-- shared/
|   |-- client/
|   |-- supplier/
|   `-- admin/
|-- tests/
|   |-- __init__.robot
|   |-- test_data.py
|   |-- auth/
|   |-- admin/
|   |-- client/
|   `-- supplier/
|-- requirements.txt
`-- .env #gitignored, contains environment secrets
```

REPUBLIC OF TUNISIA
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology

تصميم وتنفيذ مجموعة اختبارات الدخان ودمجها في خط الأنابيب كبوابة جودة للتكامل المستمر والنشر المستمر (CI/CD) لمنصة برمجية كخدمة (SaaS) متعددة الأكوان

الملخص

تُشغّل LittleBig Connection منصة برمجية كخدمة (SaaS) موزعة على أربعة أكوان تطبيقية. كانت مجموعة اختبارات الانحدار الخاصة بفريق ضمان الجودة، والمكوّنة من ٩٧٦ حالة اختبار مبنية باستخدام Robot Framework وSeleniumLibrary، تتطلب أكثر من ثماني ساعات للتنفيذ مع معدل هشاشة غير مقاس. يقدّم هذا العمل مجموعة اختبارات دخان مبنية باستخدام Robot Framework وBrowser Library المعتمدة على Playwright، تعمل كبوابة جودة في المرحلة الأولى من خط أنابيب التكامل المستمر والنشر المستمر (CI/CD) على GitLab، وتمنع تنفيذ اختبارات الانحدار عند فشل أي مسار حرج. تغطي المجموعة ١٨ اختباراً مستقلاً لأهم ٢٠ مساراً عبر مجالات Auth وAdmin وClient وSupplier، وتُنجز في حوالي دقيقتين بالتنفيذ المتوازي، أي أسرع بنحو ٢.٩ مرة مقارنة بمسارات Selenium المكافئة. كما تُنشر النتائج تلقائياً على Microsoft Teams وتُتابع عبر لوحة معلومات GitLab Pages.

الكلمات المفتاحية: Playwright، Browser Library، Robot Framework، اختبارات الدخان، الاختبارات الشاملة، التكامل المستمر والنشر المستمر (CI/CD)، GitLab، بوابة الجودة، هشاشة الاختبارات، استقلالية الاختبارات، نموذج كائن الصفحة، DevOps.

Conception, implémentation et intégration dans le pipeline d'une suite de tests smoke comme barrière qualité CI/CD pour une plateforme SaaS multi-univers

Résumé

LittleBig Connection exploite une plateforme SaaS sur quatre univers applicatifs. La suite de régression QA (976 tests sur Robot Framework et SeleniumLibrary) dépassait huit heures d'exécution avec un flakiness non mesuré. Le travail introduit une suite smoke basée sur Robot Framework et Browser Library/Playwright, exécutée en première étape du pipeline GitLab CI/CD pour bloquer la régression dès qu'un parcours critique échoue. Dix-huit tests indépendants couvrent les vingt parcours prioritaires des univers Auth, Admin, Client et Fournisseur, s'exécutent en environ deux minutes en parallèle (≈2,9× plus rapides que les parcours Selenium équivalents), avec publication des résultats sur Microsoft Teams et suivi via un tableau de bord GitLab Pages.

Mots clés : Robot Framework, Browser Library, Playwright, tests smoke, tests end-to-end, CI/CD, GitLab, barrière qualité, flakiness, indépendance des tests, page object model, DevOps.

Design, Implementation, and Pipeline Integration of a Smoke Test Suite as a CI/CD Quality Gate for a Multi-Universe SaaS Platform

Abstract

LittleBig Connection runs a SaaS platform across four application universes. The QA regression suite (976 Robot Framework/SeleniumLibrary tests) took over eight hours with an unmeasured flakiness rate. This work introduces a smoke suite built with Robot Framework and Browser Library/Playwright that runs as the first GitLab CI/CD stage and blocks regression whenever a critical path fails. Eighteen independent tests cover the twenty highest-priority journeys across Auth, Admin, Client, and Supplier, complete in about two minutes in parallel (≈2.9× faster than equivalent Selenium paths), and publish results to Microsoft Teams with tracking through a GitLab Pages dashboard.

Keywords: Robot Framework, Browser Library, Playwright, smoke testing, end-to-end testing, CI/CD, GitLab, quality gate, flakiness, test independence, page object model, DevOps.

Corporate Information

Company: AMARIS CONSULTING TUNISIA
Address: 9 NIDA Building, Lot 45, Chotrana Industrial Zone 2088, Ariana Medina 2036